



StatsPAI: Validation-Tiered Python Workflows for Causal Inference

Biaoyue Wang 
Stanford University
and StatsPAI Inc.

Scott Rozelle 
Stanford University
and StatsPAI Inc.

Abstract

StatsPAI is a Python package for applied causal inference and econometrics in which every registered function carries an evidence tier that a user or an agent can query at call time. This article asks what such a label certifies, and describes the design that makes the answer checkable. Parity rows are classified by what the **StatsPAI** side actually executes, verified by a call trace of every module (86 of 89 run native code, 3 wrap third-party Python libraries, and none call the reference they are compared against); evidence is recorded per configuration, so a user can ask whether the variant, inference option, and code path just fitted is the one that was tested; automated diagnostics distinguish a missing check from one that does not exist for the fit; and stochastic estimators are graded by seed-replicated equivalence against an explicit margin rather than by their sampling standard errors. The evidence covers a twelve-estimator core: same-byte parity with canonical R implementations (89 modules) and audited **Stata** bridges (85), known-truth recovery, Monte Carlo coverage at $B = 1,000$ reported with bias, Monte Carlo standard deviation, and standard-error calibration, and a performance study that keeps unfavourable comparisons. Applying the design to the package found defects that numerical agreement alone had not: two parity rows that compared R with itself, a coverage row that measured a different estimator from the one it named, an entropy-balancing standard error twice the estimator's Monte Carlo standard deviation, and an audit that requested an over-identification test for a just-identified model. Seven worked examples, executed verbatim and one returning the same numbers as its R and **Stata** references on identical bytes, and a licence-free replication path complete the article. The evidence for the agent-facing interface is mechanical and contractual, not a claim of behavioural agent superiority.

Keywords: causal inference, applied econometrics, validation tiers, Python, R, Stata, software, machine learning, agent-facing registry API, large language models, reproducibility.

1. Introduction

Applied causal work now spans a large and fragmented software stack: Stata commands, methodological-reference R packages, and an expanding but incomplete Python ecosystem. A single project can require OLS and IV, staggered difference-in-differences, regression discontinuity, synthetic control, matching, double machine learning, sensitivity analysis, and publication tables. The statistical literature has converged toward more explicit design checks and estimator-specific inference (Imbens and Wooldridge 2009; Angrist and Pischke 2009; Callaway and Sant’Anna 2021; Roth, Sant’Anna, Bilinski, and Poe 2023; Wing, Yozwiak, Hollingsworth, Freedman, and Simon 2024; Baker, Callaway, Cunningham, Goodman-Bacon, and Sant’Anna 2026; Rambachan and Roth 2023; Abadie, Diamond, and Hainmueller 2010; Chernozhukov, Chetverikov, Demirer, Duflo, Hansen, Newey, and Robins 2018); the software interface has not converged with it. Users still cross package boundaries, result classes, naming conventions, and diagnostic vocabularies. Large-language-model agents add a further constraint: they need typed schemas and structured errors, not prose-only help pages.

This paper introduces **StatsPAI**, a Python platform for applied econometrics and causal inference. The package provides (i) a unified `import statspai as sp` surface across common causal designs, (ii) shared reporting conventions for mature estimator results, diagnostics, citations, and export methods, (iii) a machine-readable registry that can be exposed through a Model Context Protocol (MCP) server (Anthropic 2024), and (iv) an explicit validation discipline against analytical targets, canonical R implementations, and selected Stata migration bridges. This article describes **StatsPAI** 1.30.1, distributed on PyPI (`pip install statspai`, tag v1.30.1) under the MIT licence, with source at <https://github.com/brycewang-stanford/StatsPAI>.

A short paper on the package appeared in the *Journal of Open Source Software* (Wang and Rozelle 2026). It established the unified interface, the registry and agent schemas, the structured result objects, and the existence of a same-byte cross-language parity harness and of a 1,000-replication coverage run for OLS, DiD, and IV. This article does not re-establish those facts. Its question is the one a parity harness raises but cannot answer by itself: *what exactly does a validation label certify?* A green parity row can be earned by the wrong code (a Python wrapper that calls the R reference it is compared against), vouch for the wrong function (an alias credited with evidence that ran another entry point), rest on the wrong statistical comparison (a stochastic estimator judged against its sampling standard error rather than its own Monte Carlo error), or describe a configuration the user is not running. The contribution is the design that makes those failure modes checkable, and the evidence of what applying it found. Concretely, the article adds (i) evidence provenance that is verified rather than asserted — every parity row is classified by what the **StatsPAI** side actually executes, checked by a call trace, and every inherited label must name a test that ran the inheriting function (Sections 5.3 and 5.9); (ii) statistical-applicability semantics for automated diagnostics, so that a check which does not exist for a fit is reported as not applicable rather than missing (Section 4.1); (iii) a seed-replicated standard for stochastic estimators that separates algorithmic Monte Carlo error from sampling error and states equivalence against an explicit margin (Section 5.4); (iv) a fixed adjudication procedure for disagreements, with its outcomes (Section 5.8); and (v) new evidence the JOSS paper does not contain: the estimand and identification contracts of a twelve-estimator core, configuration-level coverage diagnostics (bias, Monte Carlo SD, standard-error calibration, interval length), a performance study that reports unfavourable

comparisons, and seven executed worked examples. Applying the design to its own package found defects that numerical parity alone had not: two Honest-DiD rows that compared R with itself, a causal-forest coverage row whose entry point reported a separate AIPW estimator rather than the forest’s, an entropy-balancing standard error twice the estimator’s Monte Carlo SD, and an audit that asked for an over-identification test on a just-identified model. The registry distinguishes `certified`, `validated`, `api_stable`, and `experimental` symbols – queryable at call time – so numerical validation is an evidence tier rather than a blanket claim over the whole API surface; the full census opens Section 5.

Listing 1 fixes ideas. The classical Card (1995) returns-to-schooling design is estimated four ways — OLS, instrumental variables, double machine learning (Chernozhukov *et al.* 2018), and a causal forest (Wager and Athey 2018) — from a single import, and three of the four fitted objects, which belong to three different result classes, are rendered into one publication table by a single call. Three design properties recur throughout the paper: the estimator surface is *flat* (classical and modern methods are siblings under `sp.`); the result objects are *interoperable* (mixed estimators feed directly into one `sp.regtable` call); and the platform exposes *audit hooks* (`sp.audit`) that list the diagnostic evidence a design demands but the user has not yet supplied. Section 4.1 runs the same session at a prompt and prints what each of these calls returns.

Every listing in this paper is executed verbatim by the replication archive’s listing audit, and for the transcripts the audit goes further: it replays each one statement by statement through a real interpreter loop and compares the output printed here, line for line, with the output the code actually produces (Section 8.2). A printed transcript is quoted as evidence and read as evidence, so it is checked like evidence.

```
import statspai as sp

card = sp.datasets.card_1995()
X     = ["exper", "expersq", "black", "south", "smsa"]

ols = sp.regress("lwage ~ educ + " + " + ".join(X), data=card)
iv  = sp.iv("lwage ~ " + " + ".join(X) + " + (educ ~ nearc4)",
            data=card)
dml = sp.dml(card, y="lwage", d="educ", X=X,
             model="plr", n_folds=5, random_state=42)
cf  = sp.causal_forest("lwage ~ educ | " + " + ".join(X), data=card,
                      n_estimators=2000, discrete_treatment=False,
                      random_state=42)

table = sp.regtable(ols, iv, dml, keep=["educ", "ATE"], stars=True,
                   model_labels=["OLS", "2SLS", "DML PLR"])
table.to_docx("returns_to_schooling.docx")
```

Listing 1: **StatsPAI** reproduces and extends Card (1995) from a single import; classical and modern estimators are siblings under `sp.` and return one interoperable result contract.

The package surface is deliberately broad, spanning classical regression and panel models, the modern DiD, RD, and synthetic-control literatures, ML-based treatment-effect estimation, matching and weighting, sensitivity analysis, and publication output. Breadth, however, is not the paper’s evidentiary claim. In this paper, “validated” means a specific artifact exists:

known-truth recovery, cross-language parity on identical data bytes, Monte Carlo coverage, or an explicit convention disclosure. Unit and regression tests are API-contract evidence; they do not by themselves promote a helper into the numerical-validation tier. An `api_stable` helper has a stable call signature; it does not acquire statistical credibility by proximity to validated estimators. Table 1 makes this boundary explicit before the paper turns to architecture and validation evidence.

Layer	Examples	Evidence used in this paper
Unified causal API	OLS/IV, DiD, RD, synth, DML, forests, matching, causal-impact workflows	Shared entry points, mature result objects, exports, and worked examples
Validated causal core	The twelve-estimator validation suite	Analytical recovery, R parity, selected Stata bridges, coverage, and performance checks
Agent-facing surface	Registry schemas, agent cards, MCP tools, structured errors	Schema coverage, deterministic MCP trace, and citation/audit checks
Broader registry	Long-tail econometric helpers and optional backends	Reported as <code>api_stable</code> or <code>experimental</code> unless backed by the same evidence standard

Table 1: Scope of the submission. The package is broader than the validated core; the paper-level claim is limited to the rows backed by the artifacts in Sections 4–7.

The closest implementations cover important parts of this terrain, but with different centers of gravity; Table 2 states the comparison directly. Empirical comparisons appear later in the same-byte parity rows (Tables 8 and 9) and in the timing study (Figure 3); per-entry-point detail is generated into `Paper-JSS/parity/replacement-table.md`. The contribution is therefore an integration-and-validation layer around reference software, not a claim that **StatsPAI** supersedes specialized packages or reference implementations in their home domains, such as `fixest`, `did`, `rdrobust`, `Synth`, `synthdid`, `DoubleML`, or `grf`. Representative software citations include `Synth`, `MatchIt`, `DoubleML`, `EconML`, `CausalML`, and `DoWhy` (Abadie, Diamond, and Hainmueller 2011; Ho, Imai, King, and Stuart 2011; Bach, Chernozhukov, Kurz, and Spindler 2022; Battocchi, Dillon, Hei, Lewis, Oka, Oprescu, and Syrgkanis 2019; Chen, Harinen, Lee, Yung, and Zhao 2020; Sharma and Kiciman 2020).

The submission should therefore be read in three layers. The manuscript describes the user-facing API and the statistical-software design choices; the validation tables state which methods have parity, coverage, performance, or boundary evidence; and the replication archive contains the executable ledgers that determine whether a label such as `certified`, `validated`, or `api_stable` is justified. This separation is intentionally conservative: a broad namespace is useful only if readers can tell which parts are statistical evidence and which parts are stable software infrastructure. To keep the printed article compact, the archive also includes a reviewer evidence map and editor screening checklist that route validation-scope, reproduction, related-review and COI, sustainability, and release-boundary questions to executable artifacts. Section 2 describes the architecture the evidence attaches to and Section 3 the agent-facing interface; Section 4 runs seven worked examples, with staggered DiD as the main case study; Sections 5–7 report the validation, performance, and interface evidence.

2. Software architecture

StatsPAI is implemented in Python on top of **NumPy** (Harris *et al.* 2020), **SciPy** (Virtanen

Comparable tion	implementa- tion	Where it remains the reference choice	StatsPAI contribution and boundary
statsmodels , pyfixest	linearmodels ,	Mature regression, panel, covariance, and HDFE APIs with focused maintainer communities.	StatsPAI adds a causal-workflow layer, audit prompts, citations, and agent schemas around common calls; it is not a replacement for their full model-specific APIs or fastest specialist paths.
DoubleML , CausalML , grf	EconML ,	Deeper ML-causal and CATE-specific surfaces, tuning controls, and method-family specialization.	StatsPAI places DML and forests inside the same result, audit, and validation ledger; the forest row is only a T3 stochastic agreement claim, not deterministic parity with every grf option.
DoWhy		Graph-first identification and refutation workflows.	StatsPAI is estimator- and evidence-led: it emphasizes parity artifacts and shared output contracts, but it does not attempt to match graph-identification automation.
did , fixest , rdrobust , rd-density , Synth , synthdid , MatchIt , and Stata commands such as csdid , reghdfe , rdrobust , and sdid		Authoritative reference implementations in their home ecosystems, often with more mature edge-case options and community history.	StatsPAI validates against these references on identical data bytes where possible and records disagreements as ledger entries rather than strict parity claims; the remaining T4 disclosure is the classical-SCM identification/non-uniqueness gap on the calibrated Basque replica, while generalized SCM, RD-density, and SDID are native parity rows in 1.30.1. Stata bridges require a separate licensed runtime.
differences , and narrower packages	causalimpact , workflow	Focused workflows with less surface area for users who need exactly that model family.	StatsPAI standardizes cross-family hand-offs through one namespace and one post-estimation contract; that breadth comes with a larger dependency surface and a validation ledger that should be read row by row.

Table 2: Closest existing implementations and comparative scope, surveyed in May 2026 and spot-checked in September 2026.

et al. 2020), **pandas** (McKinney 2010), **statsmodels** (Seabold and Perktold 2010), **scikit-learn** (Pedregosa *et al.* 2011), and **linearmodels**, with optional backends kept behind extras and lazy imports. The architecture has four layers: registry, dispatcher, result object, and output pipeline.

2.1. Design principles

Seven principles govern the public API. A single flat namespace exposes routine calls through **sp.<function>**. Mature estimator paths return structured result objects rather than raw tuples, while auxiliary helpers – visualization and data-inspection utilities such as **sp.panel_view**, which returns Matplotlib handles plus an **info** dictionary rather than an estimator result – remain registered with explicit stability and validation metadata. Method families use dispatchers (**sp.synth(method=...)**, **sp.dml(model=...)**, **sp.did(...)**). Reference parity is attempted before broadening a validated claim. Failures are loud: assumption violations raise typed exceptions or warnings and record diagnostics. Output-changing changes go through the changelog and **MIGRATION.md**. Finally, every public function must be discoverable through the same registry used by human help functions and agent schemas.

2.2. Why an integration package?

A reasonable alternative is to compose **statsmodels**, **linearmodels**, **DoubleML**, **EconML**, **DoWhy**, and method-specific R or Stata bridges directly, which is often right for a single specialized model. The friction appears when an applied project moves across identification designs: diagnostics, citations, validation status, error handling, export formats, and agent schemas no longer share one contract. **StatsPAI** therefore treats wrapping as insufficient unless a method enters the common registry, result-object surface, typed-error envelope, and evidence ledger. Specialized packages remain the reference implementations; the contribution here is to make cross-family handoffs auditable and reproducible. The package is organized by identification layer rather than by the historical boundaries of R and Stata packages – each layer reports its own design diagnostics (balance and overlap, first-stage strength, cohort support and event-study paths, bandwidth and density checks, donor weights and placebos, cross-fit diagnostics) – and the common point is not shared identification assumptions but a shared post-estimation contract.

2.3. Registry and discovery layer

The central data structure is a registry of `FunctionSpec` records. Each entry carries a canonical function path, category, tags, stability and validation status, parameter metadata, result class, reference key, example call, and optional agent-card fields such as assumptions, preconditions, failure modes, alternatives, limitations, and typical sample-size guidance. The registry powers `sp.help()`, `sp.list_functions()`, `sp.search_functions()`, `sp.describe_function()`, `sp.function_schema()`, `sp.agent_card()`, the import-free `schemas/` bundle, and the MCP tool catalogue. Tests assert that exported public callables are registered and that validated/certified entries expose their evidence status.

The validation tier is therefore not a paper-only artifact: it is queryable from the running software for *every* registered function. `sp.help()` prints a per-tier census and points the user to `sp.list_functions(validation_status="certified")` for the validated core, while `sp.describe_function(name)` prints a `Stability/Validation` header with the attached evidence notes before the description, so a user or agent landing on an `api_stable` or `experimental` helper sees its trust tier before relying on its output.

```
>>> spec = sp.describe_function("callaway_santanna")
>>> spec["stability"], spec["validation_status"]
('stable', 'certified')
>>> for note in spec["validation_notes"][:3]:
...     print(note)
Track A parity seed
R parity module 04_csdid
Stata parity module 04_csdid
>>> import textwrap
>>> print(textwrap.fill(spec["limitations"][0], width=64))
clustervars is not yet supported with bstrap=False; the analytic
standard errors do not account for within-cluster dependence, so
the multiplier bootstrap is required
```

Listing 2: The validation tier is visible through the same discovery surface used by users and agents; this excerpt is generated from `sp.describe_function("callaway_santanna")`.

2.4. Family dispatchers

Family dispatchers reduce method sprawl without hiding assumptions. `sp.synth` dispatches classical, SDID, augmented, generalized, matrix-completion, penalized, and related synthetic-control variants. `sp.decompose` covers Blinder–Oaxaca, Gelbach, Fairlie, RIF, DFL, Machado–Mata, Melly, counterfactual-distribution, and related decompositions. `sp.dml` dispatches partially-linear, partially-linear IV, interactive, and LATE-style DML variants. The dispatcher contract is explicit: allowed `method=` values are enumerated in the registry schema, and validated or mature variants return compatible result surfaces so that summary, plotting, audit, citation, and export calls do not change downstream.

The same uniformity extends to inference. Across the regression family (`regress`, `feols`, `ivreg`, `panel`, `hdfe_ols`), one canonical keyword, `vce=`, spells the full standard-error menu: classical, heteroskedasticity-robust, one- and two-way clustering, bias-reduced CR2/CR3 (Pustejovsky and Tipton 2018), the wild cluster bootstrap (Cameron, Gelbach, and Miller 2008; Roodman, Nielsen, MacKinnon, and Webb 2019), and Conley spatial HAC (Conley 1999); fixed-effects Poisson and GLM additionally expose CR2/CR3 and the restricted score wild bootstrap. Each option is pinned to an external reference implementation — Stata’s `reghdfe`, `boottest`, `acreg`, and `ppmlhdfe` (Correia, Guimarães, and Zylkin 2020), and R’s `clubSandwich` and `sandwich` (Zeileis, Köll, and Graham 2020) — and a coverage matrix in the package’s continuous integration ratchets the count of natively wired, externally validated cells monotonically upward.

2.5. Result-object hierarchy

Most estimators return either `EconometricResults` or `CausalResult`; specialized classes must implement the same public surface. The standard contract covers summaries, plotting, dictionary and JSON payloads, LaTeX/DOCX export, citations, audits, diagnostics, and degradation records. This is why a mixed OLS/IV/DML/forest table can be passed to `sp.regtable` (or its R-style alias `sp.modelsummary`), and why an MCP agent can keep only a result handle rather than copying coefficient and covariance arrays between tool calls.

2.6. Performance backbone

The performance-critical path is high-dimensional fixed-effect absorption for fixed-effect and panel estimators. The submitted artifact uses the reproducible Python path built on **NumPy** and **Numba** (Lam, Pitrou, and Seibert 2015). An experimental Rust crate and selected **JAX** and **PyTorch** paths exist behind extras, but the numerical and timing claims in this paper do not depend on a compiled native extension. Optional acceleration is treated as a backend choice, not as a separate estimator or a hidden requirement.

2.7. Output pipeline

Publication output is unified through `sp.regtable`, `sp.modelsummary`, and result-level exporters. Mixed `EconometricResults` and `CausalResult` objects can be rendered as LaTeX, Markdown, HTML, Word, Excel, or JSON-safe agent payloads. `sp.outreg2` gives a Stata-like migration surface, while `sp.paper()` uses the same rendering layer to assemble tables, figures, citations, and a report.

2.8. The four-stage workflow

The layers combine into a four-stage workflow – detect (`sp.detect_design`, `sp.preflight`, `sp.recommend`), fit (the estimator dispatchers), inspect (`summary()`, `plot()`, `sp.audit()`, sensitivity wrappers), and report (`sp.regtable`, export methods, `cite()`, `sp.paper()`) – with the same boundaries available to human users and to MCP agents.

3. The agent-facing registry API

StatsPAI exposes the same surface used by a human Python user to LLM agents through typed schemas and an MCP server. The paper’s claim is mechanical and contractual: schemas, typed errors, result handles, citations, and deterministic traces make machine-orchestrated calls inspectable by the same validation ledger as human calls. It is not a behavioural claim that an autonomous agent will always choose the right empirical design. The agent layer is a software-interface contribution rather than an intelligence benchmark: it reduces four otherwise opaque handoffs – tool discovery, failure recovery, result reuse, and citation resolution – to artifacts a reviewer can inspect in the same archive, which is why the validation target is schema/MCP/citation reproducibility, not a model-quality score.

3.1. Why a schema, not a docstring

Docstrings, R help pages, and Stata help files are prose interfaces. A tool-using agent needs a typed object containing parameter names, types, defaults, required fields, enumerated choices, and a short description including stability and limitations when relevant. The schema call returns the executable OpenAI-style object; the description and agent-card calls return richer planning metadata such as assumptions, preconditions, failure modes, alternatives, typical sample size, references, and examples.

3.2. The function-schema layer

The discovery surface lists registered functions, describes a selected function, emits its executable schema, and returns curated agent cards. These functions are themselves registered, which lets an agent inspect the available estimator catalogue before selecting a method. Complementary design-detection and preflight calls expose cheap planning diagnostics before an estimator is run.

3.3. The typed exception envelope

Estimator failures inherit from `StatsPAIError` and carry structured fields: `error_kind`, `detail`, `recovery_hint`, `alternative_functions`, and any partial `diagnostics`. Warnings use the same idea for non-fatal degradations. The MCP server serializes these records as structured errors, so an agent can branch on `weak_instrument`, `no_overlap`, `rank_deficient`, or similar identifiers without scraping English messages.

3.4. The estimand-first DSL and robustness audit

`sp.causal_question(...)` accepts a requested estimand, treatment, outcome, data, optional design, covariates, instruments, and sensitivity flag, then routes to the appropriate estimator

and returns a `CausalQuestionResult`. The result bundles the fitted estimator, the design-detection provenance, optional sensitivity output, and an `audit_report`. The standalone `sp.audit(result)` call produces the same missing-evidence checklist for any fitted result and points to follow-up functions such as RD density tests, Honest-DiD sensitivity, or unobserved-confounding bounds. Each check carries one of four statuses — passed, failed, missing, or not applicable — and the last is decided from the fit’s own metadata by an explicit applicability rule (an over-identification test is not applicable to a just-identified IV model, and says why), so an agent is never sent after a test that does not exist. The companion `sp.validation_scope` answers the neighbouring question of which validation artifacts cover the fitted configuration (Section 5.4).

3.5. The bibliography verifier

Every public-facing citation emitted by the package must come from the curated `paper.bib`. Function specs store bibliography keys, result objects expose `cite(format=...)`, and tests assert that registered keys parse cleanly and do not collide on author/year/title triples. When a key is missing, the package raises an error rather than emitting a plausible-looking citation. This targets a concrete agent failure mode: hallucinated references.

3.6. The MCP server

The `statspai-mcp` server exposes registered functions as MCP tools. Tool inputs are generated from the registry schemas, outputs are JSON-safe result payloads plus `result_id` handles, and typed errors use the exception envelope above. Chained tools cover auditing, Honest-DiD sensitivity, generic sensitivity analysis, CS aggregation, Bacon decomposition, paper generation, and citation, and they accept result handles instead of requiring an agent to marshal coefficient, covariance, or cross-fit arrays through model context. The deterministic trace in Section 7 exercises this contract.

3.7. Comparison with existing platforms and prior art

Generating a JSON schema from a typed signature is solved by **Pydantic**, **FastAPI**, and the LLM-vendor tool adapters; what is package-specific here is that the schema is the statistical registry itself, tied to validation status and typed statistical errors, with result handles for estimate-to-audit-to-sensitivity chains – a validation-tiered tool catalogue with a citation verifier, which statistical help systems do not expose.

4. Worked examples

The paper uses seven worked examples as an end-to-end smoke test of the validated core rather than as seven miniature empirical papers. Each example has an executable script under `Paper-JSS/replication/` that writes JSON, table fragments, and any figures used by the build. The examples cover the identification strategies an applied econometrician is most likely to ask a unified Python front end to handle: selection on observables, IV, staggered DiD, RD, synthetic control, machine-learning treatment-effect estimation, matching, causal time-series counterfactuals, and – the case that reaches across all three ecosystems – panel counterfactual estimators and interaction effects run on the same bytes in Python, R, and Stata.

The non-trivial workflow case study is the `mpdta` staggered-DiD example: it moves from design-specific estimation to aggregation, event-study inspection, Honest-DiD sensitivity, Bacon decomposition, citations, and export from the same fitted objects. The other examples are shorter stress tests around the same contract, especially where a reference package, public data extract, or known-truth simulation gives a clean validation target. Table 3 maps those examples to the replication scripts and validation facts used by the paper.

Example	StatsPAI surface exercised	exer-	Headline validation fact	Replication artifact
Card (1995)	<code>sp.regress</code> , <code>sp.dml</code> , forest	<code>sp.iv</code> ,	OLS and 2SLS match R on <code>wooldridge::card</code> at machine precision; DML/forest are reported as stochastic ML checks.	<code>ex01_card.py</code> ; original-data module <code>01_card</code>
Basque (2003)	<code>sp.synth</code> , sensitivity		ADH special-predictor specification on the original bytes gives post-1970 gap -0.8946 vs R -0.8944 vs Stata -0.8943 ; default <i>V</i> -choice is disclosed separately.	<code>ex02_basque.py</code> ; original-data module <code>03_basque</code>
<code>mpdta</code> staggered-DiD	CS, aggregation, Honest-DiD, Bacon		Simple ATT matches <code>did</code> and <code>csdid</code> to machine precision on the calibrated replica bytes, and matches <code>did</code> at 2.3×10^{-14} on the original <code>did::mpdta</code> bytes.	<code>ex03_csdid.py</code> ; module <code>04_csdid</code>
Lee (2008) RD	RD robust and density diagnostics		The native CCT bandwidth and robust estimate match <code>rdrobust</code> on the <code>RDsenate</code> extract to 3×10^{-13} ; the official <code>rdrobust</code> Python port agrees with the native path.	<code>ex04_lee.py</code> ; modules <code>06_rd</code> , <code>09_rddensity</code>
LaLonde / Dehejia-Wahba	OLS, PSM, IPW, balance diagnostics		Original public extracts from <code>MatchIt</code> and <code>causalsens</code> match R; the observational design preserves the classic sign/bias warning.	<code>ex05_lalonde.py</code> ; original-data modules <code>04</code> , <code>04b</code>
Advertising lift	<code>sp.causal_impact</code>		Known simulated lift = 4.2 is recovered at 4.306 with the 95% interval containing truth and pre-period RMSE below the registered guard.	<code>ex06_causal_impact.py</code>
<code>fect</code> / <code>interflex</code>	<code>sp.panel_view</code> , <code>sp.fect</code> , <code>sp.interflex</code>		Three <code>fect</code> outcome models and three <code>interflex</code> estimators return the same numbers as R <code>fect</code> / <code>interflex</code> and the authors' Stata commands on identical bytes (worst relative gap 10^{-9}).	<code>ex08_fect_interflex.py</code> ; modules <code>86_fect</code> , <code>87_interflex</code>

Table 3: Worked-example coverage and replication artifacts. Each row is reproduced by the named script under `Paper-JSS/replication/`; the main text keeps to the executable contract and the headline validation fact, while full per-row provenance lives in the replication archive.

4.1. Card (1995): returns to schooling

The Card workflow (Card 1995) loads the NLSYM extract, fits OLS, 2SLS, DML, and a causal forest through the same `(y, X, data)` contract, and exports the resulting table with `sp.regtable`. `sp.datasets.card_1995()` returns the public `wooldridge::card` extract itself, so the session of Listing 1 runs on the original bytes; Listing 3 continues that session at a prompt and prints what it produces.

```
>>> print(sp.regtable(ols, iv, dml, keep=["educ", "ATE"], stars=True,
...                  model_labels=["OLS", "2SLS", "DML PLR"],
...                  rules="ascii"))
```

	OLS	2SLS	DML PLR
educ	0.0740*** (0.0035)	0.1323*** (0.0492)	
ATE			0.0741*** (0.0037)

```
=====
First-stage F                                16.72
=====
N                                3,010          3,010          3,010
R2                                0.291          0.225
Adj. R2                           0.289
F                                204.932
=====
Standard errors in parentheses
*** p<0.01, ** p<0.05, * p<0.10
>>> print(sp.audit(iv))
Reviewer audit: IV-2SLS [iv]
1 of 2 applicable checks satisfied (passed 1, failed 0, missing 1; 1 n/a)

passed weak_instrument      value 16.72 vs threshold 10
n/a    overid_test          just-identified; no restriction to test
missing anderson_rubin_ci   run sp.anderson_rubin_ci

Missing checks are evidence a reviewer will expect, not errors.
n/a checks do not exist for this fit and are not counted.
```

Listing 3: The session of Listing 1, continued at a prompt: three result classes render into one table, and `sp.audit` separates the checks this fit has passed, still lacks, or cannot have. Printed with `rules="ascii"` so the transcript is byte-safe on any console code page.

The three linear-in-educ summaries land where the literature puts them: $\hat{\beta}_{OLS} = 0.07401$ against Card’s published 0.075, and $\hat{\beta}_{IV}^{nearc4} = 0.1323$ against his 0.132, reproducing the IV-above-OLS pattern that motivates the design; cross-fit DML agrees with OLS to the third decimal. R’s `lm+sandwich` and `AER::ivreg` return the same two coefficients on the same CSV bytes at relative errors 7.9×10^{-15} and 1.6×10^{-11} (Table 9). The `sp.audit` block is the audit-hook claim made concrete, and it distinguishes three states that a flat checklist conflates. The first-stage F of 16.72 clears the $F > 10$ rule of thumb (Staiger and Stock 1997; Stock and Yogo 2005); “passed” means only that this screen was met — it is neither a test of instrument validity nor a guarantee against weak-instrument distortion, which is why weak-instrument-robust Anderson–Rubin inference is still listed as missing. The over-identification test is reported as not applicable rather than missing: with one endogenous regressor and one excluded instrument the model is just-identified, the Sargan/Hansen statistic has zero degrees of freedom, and there is no restriction to test. The audit reads the instrument and

endogenous-regressor counts from the fitted object to decide this; when an over-identified fit already carries a J statistic it is read and graded, and the text, JSON, and MCP outputs carry the same `not_applicable` status and reason.

The four estimators do not share one identification argument, and the table should not be read as four estimates of one parameter. The 2SLS coefficient uses `nearc4` as an excluded instrument and, with heterogeneous returns, identifies a local average effect for instrument-compliers under relevance, exclusion, independence, and monotonicity. The DML and forest calls in Listing 1 use no instrument: they adjust for the listed covariates and are causal only under conditional exogeneity of schooling, the assumption the IV design exists to avoid. On these data they therefore estimate an adjusted partial association, and the forest’s heterogeneity is heterogeneity in that association. The listing shows that the four fits share one result contract; it does not identify heterogeneous returns to schooling, which would require an IV-based heterogeneity estimator and its own validation. With that caveat, Listing 4 summarises the fitted forest’s conditional effects.

```
>>> print(sp.cate_summary(cf).loc[["Mean (ATE)", "Std. Dev.",
...                               "Q25", "Q75"]].round(4))
```

	CATE
Mean (ATE)	0.0765
Std. Dev.	0.0164
Q25	0.0633
Q75	0.0859

Listing 4: How far the return to schooling varies across the sample, from the forest fitted in Listing 1. With `grf`’s honest, 2,000-tree defaults the interquartile range spans roughly 6.3% to 8.6% per year around a mean of 7.7%, as R `grf` finds on the same bytes (6.3% to 8.7% around 7.6%): the estimated dispersion is modest, about a fifth of the 0.081 standard deviation the pre-1.29.0 forest engine reported for the same call on the same data.

Two boundaries are worth stating here because the rest of Section 5 depends on them. First, the Track A parity fixture for this design is a *calibrated replica*, not the extract above: it is a redistribution-safe guard that preserves the IV-greater-than-OLS pattern, which is why the Card rows of Table 8 carry different coefficients from the original-data rows of Table 9. Second, the treatment here is years of schooling, a continuous variable, so the forest’s scalar aggregate is the average partial effect built from the continuous-treatment debiased score rather than the binary AIPW aggregate that the Track B forest row validates (Section 5.5). On these bytes `cf.average_treatment_effect()` returns 0.0787 (SE 0.0049), as does `grf` 2.6.1 (0.0787, SE 0.0049); the two single fits differ by 1×10^{-5} , which is one draw from each engine rather than a grade (Section 5.4); both packages refuse the ATT and ATC targets for a continuous treatment. Both artifacts are rebuilt by `replication/scripts/ex01_card.py`; the `grf` side by `ex01_card_grf.R`.

4.2. Abadie–Gardeazabal (2003): the Basque country

The Basque example (Abadie and Gardeazabal 2003) exercises the synthetic-control dispatcher and the post-estimation sensitivity hooks. The publication-critical point is that the paper does not hide SCM non-uniqueness behind a loose tolerance: under the canonical Abadie–Diamond–Hainmueller (Abadie *et al.* 2010) setup used by `Synth::synth` – one special predictor per pre-treatment year and nested V optimization – and on the *original* `Synth::basque` bytes,

Listing 5: Callaway–Sant’Anna fit, simple- and event-study aggregation, Rambachan–Roth Honest-DiD bounds, and the Goodman–Bacon decomposition — all from the same fitted object.

`sp.callaway_santanna` estimates group-time ATTs, `sp.aggte` reports simple and dynamic aggregations, `sp.honest_did` applies the Rambachan–Roth sensitivity post-estimator, and `sp.bacon_decomposition` gives the TWFE weight diagnostic (Goodman-Bacon 2021). Listing 6 prints the two objects a reader of this literature would want to see: the aggregated effect, and how much parallel-trends violation it survives.

```
>>> print(f"{float(agg.estimate):.6f}  {float(agg.se):.6f}")
-0.032977  0.007961
>>> print(hd.round(4).to_string(index=False))
   M  ci_lower  ci_upper  rejects_zero
0.0000  -0.0546  -0.0214           True
0.0037  -0.0566  -0.0129           True
0.0074  -0.0582  -0.0017           True
0.0112  -0.0601   0.0058          False
0.0149  -0.0632   0.0106          False
0.0223  -0.0706   0.0180          False
```

Listing 6: The simple aggregation and the Rambachan–Roth smoothness bounds from the fit of Listing 5. `M` is the bound on how much the post-treatment trend may deviate from the linear extrapolation of the pre-treatment trend; `rejects_zero` is whether the robust confidence set still excludes zero at that bound.

The reading is the one Rambachan and Roth (2023) intend: the first-period effect is robust to a post-treatment trend deviation of up to about $M = 0.0074$ log points, and the confidence set admits zero once the allowed deviation reaches 0.0112. That breakdown value, not the pre-trend test’s p -value, is the quantity a referee should be given, and it comes off the same fitted object as the point estimate.

That last clause is the case for the shared contract, and it can be measured. The Rambachan–Roth interval needs the *joint* covariance of the event-study coefficients, the relative times, and the position of the reference period. Composed by hand, the workflow extracts these from one package’s aggregation object and re-enters them into another’s sensitivity function; the tempting shortcut is to rebuild the covariance from the reported standard errors. On `mpdta` the pre- and post-period coefficients are correlated up to 0.61, and the shortcut is not innocuous: with `diag(SE2)` in place of the joint covariance the breakdown value rises from 0.0080 to 0.0101, so the estimate would be reported as surviving trend deviations 26% larger than the data support, and the $M = 0$ interval shifts from $[-0.0546, -0.0214]$ to $[-0.0477, -0.0156]$. In **StatsPAI**, `sp.honest_did` reads the joint covariance, the relative times, and the aggregation weights from the fitted object through one extractor, `sp.event_study_vcov`, shared by nine event-study estimators; when a fit cannot supply a joint covariance it says so and falls back to a labelled approximation rather than fabricating one from standard errors. The single entry point also means a correctness fix applies once: the 1.30.0 repair of the cohort-share term in that covariance (Table 18) reached every estimator’s sensitivity analysis at the same time. The saving is not lines of code; it is that the covariance, the reference period, and the estimand travel with the estimate instead of being re-typed. The `random_state=0` in Listing 5

is not decoration: `sp.aggte` follows `did` in defaulting to a 1,000-draw multiplier bootstrap, so the aggregated standard error is a random quantity and a printed transcript of it is only reproducible once the draw is pinned. The parity evidence sits on two data sets, and the paper keeps them apart. On the calibrated `mpdta` replica of Track A module 04, **StatsPAI**, `did`, and `csdid` all return the simple ATT -0.0329765 in the R-joined harness and agree within the registered iterative tolerance on the analytic SE. On the *original* `did::mpdta` bytes, **StatsPAI** and `did` return the simple ATT -0.03995 at relative error 2.3×10^{-14} and the vignette-printed overall dynamic ATT -0.0772 at 1.6×10^{-14} (Table 9). The homogeneous calibrated replica additionally checks that Callaway–Sant’Anna, Sun–Abraham (Sun and Abraham 2021), BJS/imputation (Borusyak, Jaravel, and Spiess 2024), and DR-DiD (Sant’Anna and Zhao 2020) collapse to the same estimand when their identification restrictions coincide. Figure 2 is the printed event-study counterpart to Listing 5.

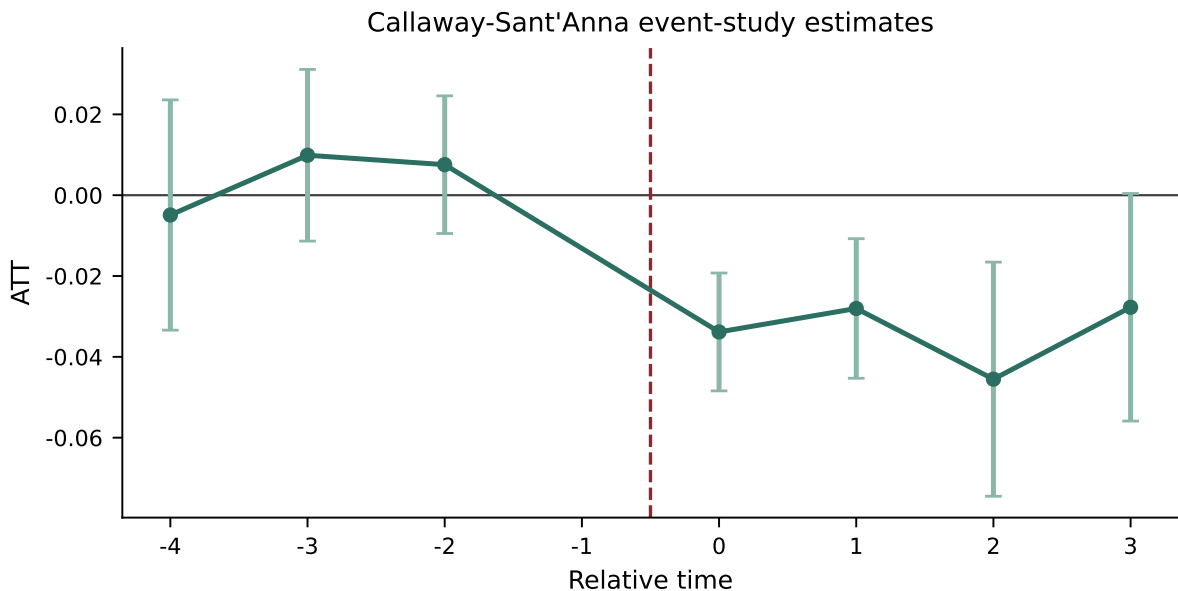


Figure 2: Callaway–Sant’Anna dynamic event study on `mpdta` (`sp.aggte(cs, type="dynamic")`): event-time ATTs with simultaneous confidence bands. Regenerated by `replication/scripts/generate_figures.py`; numerical artifacts are written by `replication/scripts/ex03_csdid.py`.

4.4. Regression discontinuity: the incumbency advantage (Lee-style sharp design)

The RD example deliberately separates the local-polynomial estimator from the bandwidth selector, and it keeps two data objects distinct to avoid over-claiming. The *exact* cross-language parity anchor is the public `rdrobust::RDsenate` extract — the U.S. Senate margin-of-victory sample distributed with `rdrobust`, a Lee-style sharp RD but *not* Lee (2008)’s original House data, which we do not redistribute. The calibrated `lee_2008_senate()` replica is a redistribution-safe drift-guard, not a parity target: on it `sp.rdrobust` returns a robust jump of 0.0763, and the external test only asserts that the estimate stays in the $[0.05, 0.12]$ structural neighbourhood. On the `RDsenate` extract the native `sp.rdrobust` default, which

implements the [Calonico, Cattaneo, and Titiunik \(2014\)](#) CCT bandwidth cascade and robust bias-corrected inference in **StatsPAI** itself (software article: [Calonico, Cattaneo, and Titiunik 2015](#)), returns the same MSE-optimal bandwidth (17.7544) and conventional and robust estimates as R’s default **rdrobust** at relative error below 3×10^{-13} (Section 5.3). Users who prefer the authors’ own code can call `sp.rdrobust(..., bwselect="cct")`, which delegates to the official **rdrobust** Python port; the two agree to 2×10^{-13} , and that agreement is recorded as a convergence check rather than counted as parity. The previous density-test selector gap is closed by the native `sp.rddensity` port (Section 5.3).

4.5. LaLonde (1986) / Dehejia–Wahba (1999): the NSW programme

The LaLonde ([LaLonde 1986](#)) / Dehejia–Wahba ([Dehejia and Wahba 1999](#)) workflow shows the same mature result surface under an experimental benchmark and an observational comparison. On `causalsens::lalonge.psid` ([Blackwell 2014](#)), the naive OLS ATT is $-\$15,200$ in both **StatsPAI** and R, and the covariate-adjusted OLS ATT is $\$751.9$ on both sides; on `MatchIt::lalonge`, 1:1 nearest-neighbour PSM agrees with **MatchIt** up to the registered caliper/default convention. The replication script also emits balance and overlap diagnostics: the point is not that matching fixes the benchmark, but that the platform keeps the experimental anchor, observational bias, and preprocessing diagnostics in one auditable pipeline.

4.6. Causal time series: advertising lift

The causal-impact example uses a deterministic Brodersen-style ([Brodersen, Gallusser, Koehler, Remy, and Scott 2015](#)) advertising-lift simulation rather than proprietary marketing data. The known post-intervention lift is 4.2 sales units per day; `sp.causal_impact` estimates 4.306 with SE 0.140, a 95% interval [4.032, 4.579], cumulative effect 129.2 against true 126.0, and pre-period RMSE 0.562. These numbers are not cross-language parity claims; they are known-truth guards that the time-series counterfactual machinery, the result object, and the publication export path are wired correctly.

4.7. Three ecosystems, one number: panel counterfactuals and interaction effects

The last example shows the same estimator returning the same number in all three languages rather than a Python replica of a published figure. It uses two method families from one research group that ship their own R and Stata implementations: the counterfactual estimators for time-series cross-sectional data of [Liu, Wang, and Xu \(2024\)](#) (**fect**; two-way fixed effects, interactive fixed effects, and matrix completion, all fitted on the untreated cells and imputing the treated ones) and the interaction-effect diagnostics of [Hainmueller, Mummolo, and Xu \(2019\)](#) (**interflex**; the linear, binning, and kernel estimators of a conditional marginal effect). **StatsPAI** ports both natively: `sp.fect` runs **fect**’s EM map step for step, and `sp.interflex` reproduces the R package’s conventions down to the linear-binning-plus-FFT density grid behind its adaptive kernel bandwidth. Listing 7 shows the workflow on the staggered two-factor panel of parity module 86: a **panelView**-style look at the treatment pattern ([Mou, Liu, and Xu 2023](#)), the three counterfactual fits, and the interaction diagnostics on the module-87 sample.

```
fig, ax, info = sp.panel_view(panel, unit="id", time="time",
                             treat="D")
```

```

info["staggered"], info["has_reversals"]      # (True, False)
fe = sp.fect(panel, y="Y", treat="D", unit="id", time="time",
             covariates=["X1", "X2"], method="fe")
ife = sp.fect(panel, y="Y", treat="D", unit="id", time="time",
             covariates=["X1", "X2"], method="ife", r=2,
             vce="bootstrap", n_boot=200, seed=0)
mc = sp.fect(panel, y="Y", treat="D", unit="id", time="time",
             covariates=["X1", "X2"], method="mc", lam=0.002)
ife.detail                                   # ATT by relative period
bins = sp.interflex(sample, y="Y", d="D", x="X", z=["Z1"],
                   estimator="binning", cutoffs=[0.3, 1.7])
bins.model_info["tests"]["p_wald"]          # linear vs binning
kern = sp.interflex(sample, y="Y", d="D", x="X", z=["Z1"],
                   estimator="kernel", bw=1.0)

```

Listing 7: Panel counterfactuals and interaction effects. `sp.panel_view` reports adoption timing, reversals, and missing cells before any model is fitted; `sp.fect` imputes the untreated outcome of every treated cell under three outcome models; `sp.interflex` estimates how the treatment effect varies with a moderator and tests the linear-interaction restriction.

One scope boundary must be stated before the numbers: the R **fect** and **interflex** packages select the factor number r , the matrix-completion penalty λ , and the kernel bandwidth by cross-validation by default, and cross-validation draws random folds. The three-way parity claim below is therefore established at *user-specified* tuning values (the listing's $r=2$, $\text{lam}=0.002$, and $\text{bw}=1.0$ are the values fixed by parity modules 86–87 so that all three implementations solve the same problem). Since 1.29.0 the selectors themselves are ported (`sp.fect(cv=True)`, `sp.interflex(cv=True)`) and graded T3 rather than T2 by construction: on identical folds the per-fold scores equal **fect**'s to 5.9×10^{-8} and **interflex**'s to 2.8×10^{-13} , and across twenty R fold seeds the selected r , λ and bandwidth agree with R's modal choice, with the CV curves inside the across-seed Monte Carlo spread (`tests/reference_parity/test_fect_interflex_cv_parity.py`). The fit at a CV-selected value equals the fit at that value supplied by hand, so the T2 rows above are unchanged by the port.

Table 4 sets the **StatsPAI** numbers next to the R and Stata goldens of the parity harness. The three calls that produce the interactive-fixed-effects row are `sp.fect(..., method="ife", r=2)`, `fect(Y ~ D + X1 + X2, method = "ife", r = 2, force = "two-way")`, and `fect Y, treat(D) unit(id) time(time) cov(X1 X2) method("ife") r(2)`; each is given the same CSV bytes, and the ATT agrees to 4×10^{-16} with R and 3×10^{-10} with Stata. The two rows that are deliberately *not* three-way are as informative as the rest. The Stata **interflex** command uses a fixed Gaussian kernel where the R package adapts the bandwidth to the moderator's density, and it tests the linear restriction without the covariate-by-bin terms and against an F reference; `sp.interflex` exposes both choices (`adaptive=`, `wald_full_moderate=`, `wald_test=`) and reproduces each side to machine precision, so the difference between the two references is a documented option rather than an unexplained gap. The complete row set (105 rows for **fect**, 25 for **interflex**, the by-period ATT paths included) is in the parity ledger.

4.8. Cross-example summary

Together, the examples exercise the same workflow repeatedly: data-loading, estimator dispatch,

Quantity	StatsPAI	R	Stata	rel(sp,R)	rel(sp,Stata)
<i>fect counterfactual estimators (module 86, staggered two-factor panel)</i>					
ATT, two-way FE (imputation)	2.500105	2.500105	2.500105	1.8×10^{-13}	9.8×10^{-10}
ATT, interactive FE, $r = 2$	2.241884	2.241884	2.241884	4.0×10^{-16}	2.8×10^{-10}
ATT, matrix completion, $\lambda = 0.002$	2.398529	2.398529	2.398529	2.8×10^{-15}	2.1×10^{-10}
<i>interflex marginal effects (module 87, binary treatment, moderator X, covariate Z)</i>					
linear ME at the grid midpoint	3.05408	3.05408	3.05408	5.8×10^{-16}	2.0×10^{-15}
binning effect, bin 1 (median $x = -0.27$)	1.51604	1.51604	1.51604	3.2×10^{-15}	5.9×10^{-16}
binning effect, bin 2 (median $x = 1.06$)	3.18212	3.18212	3.18212	2.8×10^{-16}	0
binning effect, bin 3 (median $x = 2.25$)	4.10838	4.10838	4.10838	1.1×10^{-15}	4.3×10^{-16}
Wald p-value, linear vs binning (R convention)	7.121×10^{-7}	7.121×10^{-7}	—	6.5×10^{-13}	—
Wald p-value (Stata convention: covariate-by-bin terms, F)	2.249×10^{-7}	—	2.249×10^{-7}	—	1.2×10^{-12}
kernel ME at the midpoint, adaptive bandwidth (R convention)	3.34108	3.34108	—	2.0×10^{-15}	—
kernel ME at the midpoint, fixed bandwidth (Stata convention)	3.27765	—	3.27765	—	1.1×10^{-15}

Table 4: Same bytes, three implementations. **StatsPAI** is re-estimated by `replication/scripts/ex08_fect_interflex.py`; the R (**fect** 2.4.1, **interflex** 1.4.0) and Stata (**fect_stata**, SSC **interflex**) columns are the frozen Track A goldens of modules 86 and 87. “—” marks a quantity the named reference does not report or computes under a different convention (shown on its own row).

typed result objects, diagnostics, sensitivity/audit calls, citation checks, JSON export, and manuscript table generation. The examples also enforce the paper’s central boundary: original-data rows are claimed only where the public extract can be redistributed or regenerated from a reference package, while deterministic replicas are described as structural guards on sign, scale, and workflow shape.

5. Numerical parity and statistical validity

The empirical claim of this paper is intentionally narrower than the package surface. **StatsPAI** registers 1,220 public callables, and the census behind the claim is reported once, here: 414 are **certified** and 142 **validated** (together 556), 661 are **api_stable**, and 3 are experimental. The word “validated” in this submission means that a function is in the **certified** or **validated** registry tier, and the tiers are earned in exactly two ways: **certified** by a cross-language comparison with a named reference on identical bytes, **validated** by known-truth recovery or replication of published numbers. Monte Carlo coverage rows and documented disagreements (T4) are recorded against a function but never earn a tier on their own, so a disclosure cannot be read as a demonstration. All 274 hand-written stable symbols carry parity, reference, or API-contract evidence; the remaining 568 stable auto-registered symbols are API-stable but not parity-backed, a denominator revisited in Section 9. Every **certified** symbol is linked in the registry to a named external reference implementation and the tolerance it was held to — an R or Stata parity module for all but a handful whose canonical implementation is a Python package (**econml**, **DoubleML**, **pyblp**, **obp**); those exceptions are enumerated in the package (`PYTHON_REFERENCE_ROWS`) and asserted by a test. Every **validated** symbol is linked to at least one known-truth or published-replication artifact; the validation-evidence audit in the archive fails on any certified/validated symbol whose registry entry lacks such an

attachment, so the mapping from the 89 modules to the 556 certified/validated symbols is enumerable rather than asserted.

Two properties of that census are easy to misread, so both are stated here rather than left to be recomputed. First, the certified and validated tiers answer different questions and are not summed in this paper’s coverage language. 415 symbols were compared against a named R or Stata implementation on identical inputs; 142 recover a known population parameter on a deterministic DGP or reproduce published numbers on a calibrated replica. Only the first is a parity claim. The second is the strongest evidence available for a method with no reference implementation, and reporting a combined figure would let it borrow the first one’s authority. Second, the all-registered denominator dilutes coverage: of 1,220 registered callables, 294 are result or exception classes that can never carry a parity grade and 126 are infrastructure that renders tables, draws plots, builds agent schemas or loads data. Against the 800 estimator callables that remain, cross-language coverage is 415 functions (51.9%). `sp.parity_summary()` returns this split at call time, so a reader never has to reconstruct the denominator from a printed total.

The registry tier and the parity grade are derived from one artifact, the committed parity index, through a single grade-to-tier function; a test asserts that the two agree for every stable symbol.

The validation design is a twelve-estimator suite with four evidence tracks: numerical parity (Track A), Monte Carlo coverage against known DGPs (Track B, separated into materialized B_{1000} rows and estimator-specific slow-suite B_s rows), computational scaling (Track C, Section 6), and agent-facing mechanical checks (Track D, Section 7). This is representative, not exhaustive: it covers the causal-inference core behind the paper’s statistical evidence claims, not every auxiliary helper in the registry.

Because the paper uses several graded vocabularies, Table 5 fixes them before any evidence is read. The practical reading rule is as follows. A **certified** or **validated** label is local to the named function, data contract, and evidence kind recorded in the registry; it does not promote nearby helpers or optional backends, and only a T2 row is a parity claim. Conversely, **api_stable** means that the call signature and output shape are stable enough to support user code, not that the estimator has the paper-level statistical evidence required for a validation claim.

5.1. Twelve-estimator validation suite

The reference implementations of Table 6 are cited once here: `lm` with **sandwich** (Zeileis *et al.* 2020), **AER**’s `ivreg` (Kleiber and Zeileis 2008), **fixest** (Bergé 2018), **did** (Callaway and Sant’Anna 2021), **rdrobust** (Calonico *et al.* 2014, 2015), **rddensity** (Cattaneo, Jansson, and Ma 2018, 2020), **Synth** (Abadie *et al.* 2011), **synthdid** (Arkhangelsky, Athey, Hirshberg, Imbens, and Wager 2021), **DoubleML** (Bach *et al.* 2022; Bach, Kurz, Chernozhukov, Spindler, and Klaassen 2024), **grf** (Athey, Tibshirani, and Wager 2019), and **MatchIt** (Ho *et al.* 2011), with the Stata commands `csdid` (Rios-Avila, Sant’Anna, Callaway, and Naqvi 2022), `sdid` (Clarke, Paila  ir, Athey, and Imbens 2024), `eventstudyinteract` (Sun and Abraham 2021), and `reghdfe`’s Poisson companion `ppmlhdfe` (Correia *et al.* 2020).

5.2. Estimands, identification, and inference contracts

Vocabulary	Meaning
Registry tiers	certified / validated / api_stable / experimental : the per-function evidence label queryable at call time (Section 2.3).
Evidence grades T1–T5	Kind of numerical evidence behind a row: known-truth recovery (T1), same-byte cross-language parity (T2), seed-replicated equivalence of a stochastic estimator within a stated margin (T3), documented convention or identification boundary (T4), registered but not validated here (T5) (Section 5.4).
Tracks A–D	The four evidence tracks of the validation suite: parity, coverage, performance, agent-facing mechanical checks.
B_{1000} , B_s	Monte Carlo draw counts: materialized 1,000-draw rows versus the slower estimator-specific suite.
Reviewer paths	The three reproduction routes of Section 8.2: the licence-free Python path, the pinned-R path, and the Stata path.

Table 5: Terminology used by the validation sections. Each vocabulary is orthogonal: a **certified** function can carry T2 and T4 rows on different fixtures, and any row can be rebuilt on one or more reviewer paths.

#	Method	Entry point	Reference	Data / DGP	Tracks
1	OLS + HC SE	<code>sp.regress</code>	lm+sandwich ; regress	Card 1995	A,B,D
2	2SLS	<code>sp.iv</code>	AER ; ivregress	Card 1995	A,B,D
3	HDFE	<code>sp.fast.feols</code>	fixest ; reghdfe	HDFE panel	A,B,C,D
4	Callaway–Sant’Anna	<code>sp.callaway_santanna</code>	did ; csdid	mpdta	A,B,C,D
5	Sun–Abraham	<code>sp.sun_abraham</code>	fixest ; eventstudyinteract	mpdta	A,B,D
6	Bias-corrected RD	<code>sp.rdrobust</code>	rdrobust ; rdrobust	RDsenate	A,B,D
7	RD density test	<code>sp.rddensity</code>	rddensity ; rddensity	RDsenate	A,D
8	Classical SCM	<code>sp.synth</code>	Synth ; synth	Basque; unique DGP	A,C,D
9	Synthetic DID	<code>sp.sdid</code>	synthdid ; sdid	California-99	A,B,D
10	DML PLR	<code>sp.dml</code>	DoubleML	Card/401(k)	A,B,C,D
11	Causal forest	<code>sp.causal_forest</code>	grf	Clean-overlap DGP	A,B,D
12	PSM	<code>sp.psm</code>	MatchIt ; teffects	NSW-DW	A,D

Table 6: Twelve-estimator validation suite. In the Reference column the R package precedes the Stata command. Tracks are A (parity), B (materialized 1,000-draw coverage), C (performance), and D (agent-facing mechanical checks); row 8 dispatches as `sp.synth(method="classic")` and row 10 as `sp.dml(model="plr")`. Data marked with a named extract are calibrated replicas in Track A and original extracts in Table 9. The HDFE member’s coverage row runs through the `sp.panel` fixed-effects entry of the same absorption family. Supplemental Track-B rows for 2x2 DiD and entropy balancing are reported in Table 11 but are not members of this suite.

Numerical agreement does not establish identification. This subsection therefore states the contract under which each validation-suite row has a causal interpretation. These are contracts

on the user’s design and data, not properties that software can certify from a successful function call. In particular, parity with a reference implementation can establish that two programs compute the same quantity while both remain inappropriate for a design that violates exogeneity, parallel trends, continuity, or overlap.

For the linear rows, write $Y = X\beta + u$. The OLS coefficient is always a linear-projection parameter; interpreting it causally additionally requires the relevant conditional-mean restriction. With instruments Z , 2SLS solves the sample analogue of $E[Z(Y - X\beta)] = 0$; a LATE interpretation for a binary treatment further requires relevance, independence, exclusion, and monotonicity. Absorbing fixed effects changes the projection space, not these identification requirements. The parity harness specifies the covariance estimator explicitly—HC1 or the named cluster—rather than relying on the entry point’s non-robust default.

For staggered adoption, the target primitive is

$$\text{ATT}(g, t) = E[Y_t(1) - Y_t(0) \mid G = g],$$

identified under no anticipation, a stated never-treated or not-yet-treated comparison group, conditional parallel trends, overlap, and no interference. Callaway–Sant’Anna aggregates these group–time effects; Sun–Abraham first forms cohort-by-relative-time effects and then interaction weights. A joint pre-trend test is reported, but failure to reject is not proof of parallel trends and its power must be considered.

The RD row targets the sharp/fuzzy cutoff effect (or kink contrast) under continuity, local support, and no precise manipulation; its parity row uses the native robust bias-corrected interval at the native CCT MSE-optimal bandwidth (Section 5.3).

The remaining rows target nonparametric discontinuities, counterfactual panels, or effects learned after nuisance estimation. For RD density the null is equality of the one-sided density limits at the cutoff; rejecting it is a design diagnostic, not itself an effect estimate. Classical SCM and SDID construct weighted untreated counterfactuals, so pre-treatment fit and the stability of the untreated outcome process are central. Their placebo distributions are design-based diagnostics and should not be read as generic i.i.d. sampling distributions. SDID uses placebo inference by default (with bootstrap and jackknife alternatives), while classical SCM offers in-space placebos.

DML PLR uses an orthogonal, cross-fitted score for the partially linear model $Y = \theta D + g(X) + \varepsilon$, $D = m(X) + v$. A causal reading of θ requires the corresponding conditional exogeneity and overlap conditions in addition to nuisance-rate conditions. The forest row targets $\tau(x) = E[Y(1) - Y(0) \mid X = x]$; its ATE/ATT summaries use an AIPW score and therefore require conditional exchangeability and overlap. PSM targets ATT by default and requires conditional exchangeability, common support, and a credible propensity specification. Balance and overlap diagnostics remain part of the analysis even when the reference parity row passes. The validation path uses five-fold DML cross-fitting, AIPW influence-function intervals for forest aggregates, and Abadie–Imbens heteroskedastic inference for default nearest-neighbour PSM.

5.3. Track A: Numerical parity

Track A uses three layers. First, estimators are tested against known DGP truths whenever a truth exists. Second, **StatsPAI** and the canonical R implementation read the same `%.16g` CSV bytes and write full-precision JSON. Third, when a directly comparable **Stata** command exists,

the Stata side reads the same CSV and emits a sibling JSON artifact. The current R-joined harness has 89 modules. Of those, 85 carry a canonical or audited Stata reference; each of the 4 without one records a measured, command-level reason in `compare.py::STATA_SKIP_REASON`. Rows whose defaults differ for a documented reason are labelled as convention gaps rather than as ordinary passes. Table 7 summarizes the source-tree test layers behind these claims.

Layer	Harness	Tests	Role
Analytical/reference recovery	re-reference parity	4,285	Deterministic-DGP recovery, frozen R / Stata reference fixtures on identical bytes, and cross-estimator agreement; includes the 203 tests that pin Stata’s <code>vce()</code> grammar (27 estimators, 79 estimator-by-covariance cells at 10^{-6}) and its <code>test</code> , <code>lincom</code> and <code>margins</code> post-estimation commands
Published/replica parity	external parity	88	Pinned values for Card, Lee, Basque, NSW, <code>mpdta</code> , paper-level formulas, and the four DoubleML model-class pins
Monte Carlo coverage	coverage MC	32	Slow-marked headline DGP checks, robustness rows, and one smoke row

Table 7: Validation test layers available in the source tree used for this submission. Counts are the tests `pytest` collects in each harness, reported by `sp.validation_report(collect_tests=True)` and pinned as floors by `tests/test_jss_validation_api.py`; the Track A modules of Appendix A are a separate, module-level artifact.

Table 8 is only a compact slice; the full row-level ledger is generated in the replication archive. Its fixtures are calibrated, redistribution-safe replicas (the Data column names them as such); original public extracts are handled separately in Table 9. Of the 89 R-joined modules, 88 receive a pass-type verdict, but two non-T2 Track A rows remain explicitly visible in the parity artifacts: classical SCM on the calibrated Basque replica is a T4 identification/reference-disagreement disclosure, and causal forest is a stochastic T3 equivalence claim rather than a deterministic T2 claim. A parity row is evidence about a **StatsPAI** algorithm only if the Python side ran one, so every module is also classified by what its **StatsPAI** side executes, and the classification is not taken on trust: a call-trace audit (`scripts/trace_parity_provenance.py`) runs each module under a profiler that records every call from **StatsPAI** into a package other than the numerical substrate (**NumPy**, **SciPy**, **pandas**) and every subprocess it launches, and a contract test fails if the registered classification and the trace disagree or if a module has changed since it was traced. Of the 89 modules, 86 run a **StatsPAI**-native implementation (**scikit-learn** appears only as a user-chosen nuisance learner); 3 are served by a third-party Python estimation library that **StatsPAI** wraps — panel fixed and random effects through **linearmodels**, ARIMA through **statsmodels**, and fixed-effects GLMs through **pyfixest** — and are marked as such in Appendix A, because they validate the wrapper’s argument mapping and conventions rather than an independent algorithm; 0 rely on an official port; and 0 call a reference implementation. That last count was two before release 1.31: until then the two Honest-DiD modules called `backend="honestdid"`, which hands the computation to the R

Estimator	Statistic	Data	StatsPAI	R ref.	Stata ref.	Max rel. err.	Verdict
sp.fast.feols	$\hat{\beta}_{x1}$	2-way HDFE, $N=10^4$	1.999503980	1.999503980	1.999503980	2.9×10^{-15}	T2; fixest ssc
sp.regress	$\hat{\beta}_{educ}$	Card replica	0.109986249	0.109986249	0.109986249	6.3×10^{-13}	T2
sp.iv	$\hat{\beta}_{educ}$	Card replica	0.141834409	0.141834409	0.141834409	1.1×10^{-11}	T2
sp.callaway_santanna	simple ATT	mpdta replica	-0.032976514	-0.032976514	-0.032976514	1.3×10^{-15}	T2
sp.rdrobust	robust RD, default CCT	RDsenate	7.506502484	7.506502484	7.506502483	9.4×10^{-11}	T2; native, default h
sp.dml	$\hat{\theta}_{DML}$	Card replica	0.110228260	0.110228260	0.110228260	3.7×10^{-15}	T2; ddml bridge
sp.causal_forest	AIPW ATE	clean- overlap DGP	1.012009376	1.011741552	—	0.000265	T3; seed- replicated, operator exact
sp.psm	ATT	NSW– DW replica	2277.690721	2277.690721	2277.690721	1.2×10^{-15}	T2; SE convention
sp.synth (unique)	avg post gap	identified SCM DGP	2.000000155	1.999999999	2.000000000	7.8×10^{-8}	T1+T2; identified DGP
sp.synth (classic)	avg post gap	Basque replica	-0.704244148	-0.687789210	-0.703950536	0.0239	T4; reference disagreement
sp.frontier	intercept	frontier DGP	2.071558153	2.071558166	2.071558168	7.2×10^{-9}	T2
sp.decompose	Oaxaca gap	Oaxaca DGP	0.221231026	0.221231026	0.221231026	6.3×10^{-16}	T2

Table 8: Representative Track A cross-language parity rows compiled from the same JSON artifacts as the supplementary parity ledger. Every row is inside its registered module-level tolerance in `tests/r_parity/compare.py`, which is 10^{-6} except where the verdict says otherwise: the DML row’s tighter 10^{-10} budget reflects its fully deterministic linear-learner pipeline, the forest row is a single draw from each engine whose interpretation rests on the seed-replicated comparison of Section 5.4, and the classical-SCM row is the 5×10^{-2} T4 disclosure. Data marked “replica” are the calibrated redistribution-safe fixtures; the Card-replica rows use the specification `l wage ~ educ + exper + expersq + black + south + smsa` with HC1 errors, which is why their coefficients differ from the shorter original-extract specification of Table 9.

package itself, so their rows compared R with R; the RD headline delegated to the official **rdrobust** Python port. All three now exercise native code, and each carries evidence of its own. The native smoothness FLCI is compared with **HonestDiD** after the package’s simulated folded-normal quantile is replaced by the exact one (the shipped package sits 3×10^{-4} away, which is that simulation error): the bounds agree to 1.4×10^{-8} for $M > 0$, and at $M = 0$, where the interval has a closed form (the GLS linear extrapolation of the pre-period coefficients), the native solver is within 2×10^{-9} of it while **HonestDiD**’s cone solver is 5.7×10^{-6} away. The native relative-magnitudes set is a port of the Andrews–Roth–Pakes conditional test (Andrews, Roth, and Pakes 2023) inverted over **HonestDiD**’s θ grid; on sixteen cases (diagonal and correlated covariances, a non-basis target, a single post-period) it returns the *same* accepted grid points as **HonestDiD**, and the least-favourable hybrid, whose first-stage critical value is simulated, agrees to within one grid step. The native RD default selector returns **rdrobust**’s

MSE-optimal bandwidth and robust estimate on the RDsenate bytes at relative error below 3×10^{-13} , with the port recorded only as a convergence check. Module 52 separately certifies the native classical-SCM solver on a uniquely identified DGP. A generated ledger in the replication archive classifies the remaining T4 row and fails the audit if a methodological gap appears without an explicit reviewer-risk label and promotion path; the causal-forest row is governed by the seed-replicated comparison of Section 5.4.

Native `sp.sdid` matches `synthdid`'s ATT at relative error 2.6×10^{-15} and native `sp.rddensity` matches `rddensity`'s p-value at 3.3×10^{-11} (Arkhangelsky *et al.* 2021; Cattaneo *et al.* 2020).

Headline original-data rows. The Track A fixtures above are calibrated, redistribution-safe replicas; the complementary original-data ledger runs **StatsPAI** and the canonical R reference on the *original* public extracts shipped by R packages. It currently holds 12 modules and 35 statistic rows, including six epidemiology-side modules that reproduce the NHEFS worked examples of Hernán and Robins' *What If* (IPW, g-formula, g-estimation, outcome regression, survival, and E-values). Table 9 shows the most reviewer-relevant cases; the full report is generated by `tests/orig_parity/compare_orig.py`, and the printed table is itself generated from those artifacts. The `did::mpdta` simple-ATT row makes the replica/original boundary concrete: the calibrated replica used by Track A module 04 returns -0.0330 on all three sides, while the original bytes return -0.03995 ; both are same-byte parity facts, and neither number is quoted against the other data set anywhere in this paper.

Extract	Statistic	Published	StatsPAI	R	Note
<code>wooldridge::card</code>	OLS educ	0.075	0.07401	0.07401	rel(sp,R) 7.9×10^{-15}
<code>wooldridge::card</code>	2SLS educ	0.132	0.1323	0.1323	rel(sp,R) 1.6×10^{-11}
<code>did::mpdta</code>	CS simple ATT	–	-0.03995	-0.03995	rel(sp,R) 2.3×10^{-14} ; no printed vignette anchor
<code>did::mpdta</code>	CS dynamic overall ATT	-0.0772	-0.07724	-0.07724	rel(sp,R) 1.6×10^{-14} ; vignette prints -0.0772
<code>Synth::basque</code>	ADH avg gap	-0.855	-0.8946	-0.8944	rel(sp,R) 1.8×10^{-4} ; Stata <code>synth</code> -0.8943 on the same bytes
<code>rdrobust::RDsenate</code>	robust RD jump	–	7.507	7.507	rel(sp,R) 3.5×10^{-16} ; <code>bwselect="cct"</code> path
<code>causalsens::lalonge.psid</code>	adjusted OLS ATT	700	751.9	751.9	rel(sp,R) 2.2×10^{-13} ; public composite extract
NHEFS (<i>What If</i>)	IP-weighted ATE	3.4	3.441	3.441	rel(sp,R) 1.3×10^{-11}
NHEFS (<i>What If</i>)	g-formula ATE	3.5	3.463	3.463	rel(sp,R) 7.4×10^{-12}

Table 9: Selected original-data parity rows on public extracts, generated from `tests/orig_parity/results/`. All R references read the same CSV bytes as **StatsPAI**. The *Published* column is a heterogeneous anchor — an original-paper table value (Card, Basque), a printed-vignette value (the `did` dynamic ATT), a textbook program output (NHEFS), or a rounded composite-extract value (LaLonde) — shown for context, not as a parity target; the parity claim is the **StatsPAI**-versus-R pair in the *Note* column. The `did::mpdta` simple-ATT row has no printed vignette anchor, so its Published cell is empty by design.

5.4. Evidence tiers and hard cases

Evidence is attached to configurations. A tier is attached to a function, but every artifact exercises one configuration: a method variant, an inference option, a data condition, a

code path, and an output. The Track B DML row is the plain example: the suite member is the partially linear model, and until this revision the only materialised coverage row ran the interactive model, which says nothing about the other’s intervals (a PLR row is now included, Table 11). For the twelve suite estimators the package therefore records, per artifact, the configuration it exercises, and `sp.validation_scope(result)` reads the configuration of a fitted object and returns the artifacts that match it exactly (“covered” requires a T1 or T2 row; “stochastic only” means only T3, T4, or coverage rows match; otherwise “not covered”), together with the artifacts that differ in one dimension. The map is deliberately narrower than the registry — an option it does not list is reported as not covered — and it found a gap in this paper’s own first example: `sp.iv`’s default classical 2SLS covariance, printed in Listing 3, had no deterministic reference row (Track A pins the HC1 variant). It is now pinned against `AER::ivreg` on the original Card extract, with classical, HC1, and clustered errors in just- and over-identified models, at 8×10^{-11} .

Evidence grades. The grades are those of Table 5; T5 marks a function registered but not validated in this paper. Two rows need more than a tolerance to read correctly.

Causal forest: which Monte Carlo error? For a stochastic estimator two uncertainties must be kept apart: sampling variation across datasets, which the reported standard error of the ATE estimates, and the algorithm’s own Monte Carlo variation across seeds on *fixed* data. “Agreement within combined Monte Carlo error” is a statement about the second, so it cannot be tested by dividing the gap between one **StatsPAI** fit and one **grf** fit by their sampling standard errors — which is how an earlier version of this row was graded. The forest evidence now separates three tasks. *Truth recovery and inference* are Track B’s (the AIPW coverage row below). *Estimand agreement* holds by construction: both engines report the doubly-robust AIPW average (Athey et al. 2019), and given the same forest the AIPW operator agrees with **grf**’s to machine precision. *Algorithmic agreement* is tested by refitting each engine under 50 seeds (20 at 8,000 trees) on fixed data, at 500, 2,000, and 8,000 trees, on two datasets: the reference-parity fixture and the Track A module-13 design, the latter run after the equivalence margin was fixed. Seeds are spaced 10^5 apart on both sides, because **grf** forests grown from consecutive seeds share most of their random draws: a first run with consecutive R seeds understated **grf**’s seed-to-seed spread several-fold and made **StatsPAI**’s forest look several times noisier than it is. Table 10 reports the result. At every tree count on both datasets a two-one-sided test establishes equivalence within $\delta = 0.1$ sampling standard errors — a difference that moves a 95% interval endpoint by at most 5% of its half-width — with an upper bound of at most 0.056. The two engines are equally noisy: their seed-to-seed standard deviations differ by a factor between 0.78 and 1.48 and fall with the number of trees, from 13–18% of the sampling standard error at 500 trees to 6–8% at the default 2,000. The claim is equivalence, not identity: the engines are independent implementations, and at 8,000 trees, where Monte Carlo error is smallest, their seed means differ by up to 0.025 sampling standard errors ($|z| \leq 2.6$). The single-fit Track A row is one draw from each distribution, not a parity fact on its own. Artifacts: `tests/reference_parity/test_grf_seed_mc_equivalence.py` and its fixtures.

Synthetic control: one specification, two data sets. The classical-SCM row is a T4 disclosure because the outer V optimization is non-convex and donor weights need not be

Data	Trees	Seeds	Gap	MC SE	Gap/SE _s	SD _{MC} (StatsPAI)	SD _{MC} (grf)	Equiv.
Reference fixture	500	50/50	-0.00028	2.0×10^{-3}	-0.004	1.0×10^{-2}	1.0×10^{-2}	yes
	2000	50/50	+0.00135	9.2×10^{-4}	+0.018	4.4×10^{-3}	4.9×10^{-3}	yes
	8000	20/20	+0.00139	6.2×10^{-4}	+0.019	2.3×10^{-3}	1.5×10^{-3}	yes
Module-13 design	500	50/50	+0.00003	9.8×10^{-4}	+0.001	4.7×10^{-3}	5.1×10^{-3}	yes
	2000	50/50	+0.00015	4.6×10^{-4}	+0.004	2.2×10^{-3}	2.4×10^{-3}	yes
	8000	20/20	-0.00003	3.7×10^{-4}	-0.001	1.0×10^{-3}	1.3×10^{-3}	yes

Table 10: Seed-replicated comparison of `sp.causal_forest` and `grf` 2.6.1 on fixed data (AIPW ATE; ATT in the archive). “Gap” is the difference of seed means; “MC SE” its combined algorithmic Monte Carlo standard error; “Gap/SE_s” the gap in units of `grf`’s reported sampling standard error; “MC SD” the seed-to-seed standard deviation of each engine. Equivalence (TOST, 5%) against $\delta = 0.1\text{SE}_s$ holds in every row. Generated by `replication/scripts/generate_forest_seed_table.py`.

unique — and how much that matters turns out to be data-dependent, which the ledger reports rather than averages away. On the *calibrated Basque replica* of Track A module 07 (the same ADH special-predictor specification and CSV bytes on all three sides), native **StatsPAI** tracks Stata `synth` for the average post-treatment gap at relative error 4.2×10^{-4} , while R **Synth** and Stata `synth` differ from each other by 2.3×10^{-2} . On the *original Synth::basque* bytes under the same specification, the three implementations essentially reconvene: **StatsPAI** -0.8946 , R -0.8944 , Stata -0.8943 , pairwise within 4×10^{-4} (Table 9) — still above the 10^{-6} strict-parity gate, consistent with solver tolerance on a non-convex outer problem, but far from the replica’s split. The same specification therefore admits a near-common optimum on one data set and measurably different local optima on another; a validation ledger that pooled the two facts into one tolerance would hide exactly the sensitivity a practitioner needs to know about. The solver is separately certified on an identified SCM DGP (Track A module 52): when the treated unit is exactly a unique convex combination of donors, **StatsPAI**, **Synth**, and Stata `synth` agree on the average post-treatment gap at relative error 7.8×10^{-8} and on the donor weights at 5.6×10^{-7} , the tolerance of the constrained quadratic solver. Thus the paper claims solver correctness on identified problems and documents the replica’s reference-disagreement/local-optimum problem without treating it as a strict R/Synth parity pass.

Native `sp.augsynth` now ports `augsynth`’s Ridge+SCM convention (Ben-Michael, Feller, and Rothstein 2021) (ATT at 7.9×10^{-6} on the Basque fixture). Native `sp.gsynth` now ports `gsynth`’s two-way factor convention (Xu 2017), matching it to machine precision.

5.5. Track B: Monte Carlo coverage

Track B checks 95% interval coverage on known DGPs, and reports with each rate what is needed to explain it: the estimator’s bias, its Monte Carlo SD, the mean reported standard error, their ratio, and the interval length (Table 11). The artifact holds twelve $B = 1,000$ rows; ten correspond to suite members (OLS, 2SLS, fixed effects through `sp.panel`, Callaway–Sant’Anna, Sun–Abraham, sharp RD, SDID, DML PLR, and the causal forest), and 2x2 DiD, entropy balancing, and the interactive DML model are supplemental. Three suite members have no coverage row by construction: the RD density test is a diagnostic, classical SCM has no analytic interval, and PSM’s inference evidence is the Abadie–Imbens SE parity row.

Most rows are calibrated in the direct sense: the mean standard error is within 3% of the Monte Carlo SD and coverage sits in the 99% Wilson band around 0.95. The rows that are not are explained rather than graded. *Sharp RD* (0.934) and *SDID* (0.928) have calibrated standard errors (ratios 0.98 and 0.99), so the shortfall is in the shape of the sampling distribution, not the SE. For RD this is the reference procedure’s own finite-sample behaviour: on the first 200 draws R **rdrobust** 4.0.0 returns the same intervals as **StatsPAI** to 6×10^{-12} and covers in exactly the same draws (`tests/coverage_monte_carlo/mechanisms/rd_drawwise_reference.py`). *DML PLR* under-covers clearly (0.883), and the diagnostics say why: its bias is half a Monte Carlo SD while the SE is nearly calibrated (0.88). `sp.dml` returns **DoubleML**’s numbers to 10^{-16} on the same folds and learners, so this is the procedure with the default learner, not the code. Varying one thing at a time on the same design (`tests/coverage_monte_carlo/mechanisms/dml_plr_learners.py`, $B = 500$) locates it in the nuisance fit: with the true nuisance functions coverage is 0.952 and the bias vanishes; with a random forest or a spline Lasso it is 0.940 and 0.938; with the default gradient boosting it is 0.892 at $n = 500$ and 0.936 at $n = 2,000$, where the bias is still 0.41 SD. DML’s orthogonal score makes nuisance error second order, not zero, and the default learner’s regularisation bias on this design shrinks too slowly to be negligible at these sample sizes; the result is stated in `sp.dml`’s documentation and in the configuration map. *Entropy balancing* over-covered at 1.000 in the previous revision because its standard error held the weights fixed and was twice the Monte Carlo SD; with the M-estimation variance it is now 0.945 with a ratio of 0.97 (Section 5.10). The *causal-forest* row is new in its present form. It previously over-covered (0.977, standard-error ratio 1.13) because its entry point reported a separate AIPW estimator rather than the forest’s (Section 5.9); recomputed on the forest’s own doubly-robust average at the default 2,000 trees it covers 0.959 with a ratio of 1.04, and a 300-draw check at 300 trees gives 0.973 (`tests/coverage_monte_carlo/mechanisms/forest_trees.py`). Table 12 records stress cases as failure-mode evidence rather than calibration results.

The size/power sweep closes a weakness of coverage-only reporting: an interval that never rejects can cover well without being useful. At the tested null, 2x2 DiD is conservative (size 0.024) and sharp RD rejects slightly too often (0.076 at $B = 500$), consistent with its coverage; the other rows are within Monte Carlo error of 0.05, including entropy balancing, whose size was 0.000 under the old standard error. Each curve is monotone on its own effect grid and reaches at least 0.90 at the largest effect; effect grids and DGPs differ, so this is not a cross-estimator ranking. DML, the forest, and SDID have no packaged power sweep, and the paper makes no power claim for them. Full rejection probabilities are in `tests/coverage_monte_carlo/results_b1000/size_power_b1000.json`.

5.6. Cross-estimator parity

The test suite also asserts equivalence across estimators that should collapse to the same estimand on a given DGP. For example, the Callaway–Sant’Anna, Sun–Abraham, multi-period DiD, and BJS/imputation implementations must agree on a homogeneous staggered-DiD DGP to within a multiple of $\sqrt{SE_1^2 + SE_2^2}$. Two estimators fitted to the same data are positively correlated, so this bound is loose: it is a screen for gross disagreement, not an equivalence test. It is still a useful adversarial check, because a shared bug can pass one-reference parity but fail cross-estimator agreement.

Estimator	DGP	Cover.	Bias	MC SD	Mean SE	SE/SD	CI len.
<code>sp.regress</code> (HC1)	RCT with covariates	0.952	-0.0028	0.0707	0.0708	1.00	0.277
<code>sp.regress</code> 2x2 DiD	2-period	0.955	+0.0017	0.0974	0.0999	1.03	0.392
	homogeneous DiD						
<code>sp.ivreg</code> (HC1)	strong binary-Z IV	0.962	-0.0041	0.3687	0.3627	0.98	1.422
<code>sp.callaway_santanna</code>	homogeneous	0.947	-0.0007	0.1080	0.1090	1.01	0.427
	staggered ATT						
<code>sp.sun_abraham</code>	homogeneous	0.950	-0.0007	0.1214	0.1234	1.02	0.484
	staggered ATT						
<code>sp.panel</code> two-way FE	known-coefficient FE	0.948	-0.0024	0.1245	0.1233	0.99	0.486
	panel						
<code>sp.rdrobust</code> sharp	known-jump RD, robust CI	0.934	-0.0061	0.1222	0.1201	0.98	0.471
<code>sp.sdid</code> (placebo SE)	factor-model panel, 1 treated	0.928	+0.0003	0.3095	0.3071	0.99	1.204
<code>sp.ebalance</code>	CIA, ATT (M-estimation SE)	0.945	-0.0014	0.0799	0.0774	0.97	0.303
<code>sp.dml</code> (IRM)	binary- D CIA, IRM ATE	0.968	-0.0074	0.1582	0.1653	1.04	0.648
<code>sp.dml</code> (PLR)	continuous- D PLR, default learners	0.883	-0.0261	0.0533	0.0472	0.88	0.185
<code>sp.causal_forest</code> (AIPW)	binary- D CIA, population ATE	0.959	+0.0095	0.0934	0.0968	1.04	0.379

Table 11: Track B materialized coverage artifacts, all at $B = 1,000$. Next to the coverage rate: the estimator’s bias, its Monte Carlo SD, the mean reported SE, their ratio (SE calibration: 1 is exact, above 1 conservative), and the mean interval length. The 99% Wilson reference band around nominal 0.95 is approximately $[0.935, 0.967]$. `sp.ivreg` is the same estimator as the `sp.iv` alias of Table 6. The two DML rows are distinct configurations: the PLR row is the suite member; the IRM row (binary treatment, through `sp.causal_question`) is reported separately and does not stand in for it. The forest row reports the fitted forest’s own doubly-robust ATE at the default 2,000 trees.

Estimator	Stressor	Coverage	Interpretation
<code>sp.ivreg</code>	weak instrument, median $F \approx 2.5$ ($B = 1,000$)	0.882	Expected under-coverage; preflight warns and suggests LIML/AR.
<code>sp.callaway_santanna</code>	heterogeneous timing and magnitude ($B = 1,000$)	0.946	Post-fix influence-function scaling remains near nominal.
<code>sp.causal_forest</code>	severe overlap loss ($B = 300$)	0.900	Under-covers as propensities approach 0 and 1; audit flags overlap before interpretation.

Table 12: Track B robustness rows. These are descriptive failure-mode checks, not pass/fail calibration claims.

5.7. High- N analytical parity and pinned fixtures

High- N fixtures tighten selected known-truth checks to two standard errors at $N = 5,000$ and $N = 8,000$. Pinned fixtures lock the paper replica values in Table 9 to four decimals.

Honest-DiD is pinned directly to the closed-form breakdown value in [Rambachan and Roth \(2023\)](#), and the DML/HDFE slices check influence functions and residual-regression identities independently of any single reference implementation.

Event-study paths. Value comparison is the wrong test for an event-study *path*, because implementations differ in what each half of the path is differenced against ([Roth 2026](#)). `sp.event_study_convention()` records that choice per estimator, and running it closed a defect: `sp.did_imputation` documented Stata’s `pretrends(k)` but shipped a residual construction whose leads equal the symmetric benchmark times N_0/N ([Li and Strezhnev 2025](#)), so a violated assumption read as satisfied. The default now reproduces Stata’s leads and their standard errors to 1.2×10^{-12} and 8×10^{-13} .

5.8. Adjudicating a disagreement

A parity ledger is only as credible as its procedure for the rows that do *not* agree, so the procedure is fixed in advance and every registered tolerance carries the outcome. A disagreement on identical bytes is assumed to be a **StatsPAI** defect until the first diverging intermediate quantity – design matrix, weights, residuals, sandwich meat – has been located. If the divergence is a documented choice on both sides (a degrees-of-freedom divisor, a small-sample cluster correction, an analytic versus influence-function variance), the default follows the method’s own authors, the alternative is exposed as a keyword, and the gap is registered with its mechanism and bounded by the tolerance. If the reference itself is the problem or the solution is not unique, the row is graded T4 and must carry independent evidence (a second reference, an analytic identity, or a uniquely identified DGP). If exact agreement is impossible by construction (forest randomization, bootstrap), the row is graded T3, which requires refitting on fixed data under many seeds and an equivalence margin stated against the sampling uncertainty; a single draw per side is reported but does not earn the grade. A gap that cannot be located is not called parity: it is graded “unexplained” in the tolerance audit and reported here as an open item. Two contract tests enforce the procedure mechanically: every R-joined standard-error row of every passing module must sit inside its module’s registered budget, and every Stata standard-error row that does not must be registered with a reconstruction of the Stata number. In release 1.30.1 all 89 modules pass the first test, and the second holds three entries, each reproduced from **StatsPAI**’s own quantities: the group-average SE of Stata `csdid` (the fixed-cohort-share aggregation of the joint influence functions, 10^{-14} ; module 04), the PLIV degrees-of-freedom factor ($N/(N - K)$) of Stata’s `ddml` double-machine-learning command (module 71), and the per-horizon K of Stata’s `lpdid` local projections DiD command (module 83). The Sun–Abraham row is the procedure’s worked example: it was carried as an open 0.7–2.2% item across two releases rather than absorbed into a wider tolerance, and the bisection that closed it in 1.24.0 found a defect on the **StatsPAI** side (Table 18).

5.9. Auditing the attachment, not only the number

A parity harness checks that two implementations produce the same number. It does not check the claim that sits one level above: that the function a reader calls is the function the harness ran. Track A modules are written against one entry point, while a package of this size exposes dispatchers, thin wrappers and convenience names that are *said* to reach the same

estimator core. Every such name inherits a grade it did not earn on its own, and inheritance is where an unfalsifiable claim can hide – the numbers in the ledger stay correct while the set of functions they vouch for quietly grows.

Auditing those attachments in release 1.26.0 found that, of eleven registered alias attachments, seven cited a module that never called the aliased name. Each surviving attachment now names a test that runs the alias against the canonical entry point on the module’s own bytes. One did not survive: `sp.wooldridge_did`, credited with the ETWFE module as an alias of `sp.etwfe`, returns an ATT 6.1% away on that module’s bytes because it is a different estimator; the alias was withdrawn and the function demoted to the grade its own evidence supports.

The same failure mode appeared in this revision in a coverage row. Track B’s causal-forest row ran through `sp.causal_question()`, whose forest design fitted the forest and then reported a separate cross-fit AIPW estimator with its own gradient-boosting nuisances: refitting with 30, 300, and 2,000 trees gave bit-identical results. It now reports the forest’s own doubly-robust average, identical to `cf.average_treatment_effect()`, and the row was recomputed.

5.10. Correctness history and residual risk

Every output-changing fix is pinned by a regression test and listed, with its version, in Appendix B. Two entries explain the design better than the rest. The 1.23.0 fix shows what a reference comparison catches that unit tests do not: `sp.did(..., weights=)` never forwarded the weights to a staggered branch and silently returned the unweighted estimand, which only `did::att_gt(weightsname=)` on identical bytes exposed. The 1.25.0 fix shows the limit of a parity ledger: the causal-forest aggregation was wrong for a *continuous* treatment, a path no Track A or Track B row exercised, and it surfaced in a worked example. A ledger validates the configurations it runs and is silent about the rest.

The current release controls that residual risk in three ways rather than by pointing to the number of past fixes. First, the evidence for a fitted configuration is queryable: given a result, `sp.validation_scope` reports which artifacts exercise exactly the variant, inference option, data condition, and code path of that fit, and reports “not covered” rather than inferring coverage from a neighbouring configuration (Section 5.4). Second, the release is frozen: the paper describes tag `v1.30.1`, the archive carries the source snapshot and every artifact hash, and changes after the tag cannot alter a number printed here. Third, every recorded artifact can be rebuilt from its entry point (Table 16).

Documented default-convention divergences

The remaining disclosed boundaries are classical SCM’s non-unique V, W solutions on the calibrated Basque replica (Section 5.4) and the Stata bridge, which is migration evidence with frozen JSON and do-files and requires a licence only for re-execution.

6. Computational performance

This section reports the measured Track C benchmark for the four estimators marked C in Table 6: high-dimensional fixed effects (`sp.fast.feols`), Callaway–Sant’Anna staggered DiD (`sp.callaway_santanna`), classical synthetic control (`sp.synth(method="classic")`),

and double machine learning (`sp.dml(model="plr")`). The goal is not to declare universal speed superiority over the reference implementations. It is to certify that the unified Python/**NumPy**/**Numba** stack is fast enough for routine applied work, to identify where the R reference remains faster, and to make the performance tradeoffs reproducible from the source tree.

6.1. Benchmark protocol

The benchmark harness lives under `tests/perf/`. Each module has a Python script and a companion R script that construct identical seed-fixed synthetic data-generating processes and record median wall-clock time after one warmup iteration. The three econometric rows use their canonical R packages; the DML row uses **DoubleML** Python so language-runtime and **R6** dispatch do not dominate a cheap-linear-learner comparison. All rows were measured on one machine (Apple Silicon arm64, 8 physical cores, 24 GB unified memory, macOS 26), with **StatsPAI** 1.30.1 on the Python side, after the 1-minute load average had stayed below 1.5 for five minutes. The Python-side result files record the **StatsPAI** version and source commit and the load at the start of each module (1.35 to 1.67 on eight cores); the reference-side files record the R and reference-package versions (**fixest** 0.14.0, **did** 2.3.0, **Synth** 1.1.10), so a timing can be tied to the release and conditions it describes. The full per-repetition JSON files are stored under `tests/perf/results/`; `tests/perf/compare_perf.py` regenerates Table 13 and Figure 3.

All Track-C timings in this paper are CPU timings, because the reproducibility target is hardware that applied users and reviewers can obtain without a dedicated accelerator. **StatsPAI** exposes accelerator-ready paths for selected workloads (PyTorch CUDA/MPS for the neural-causal estimators, a JAX backend for the HDFE residualizer, and `sp.fast.feols_jax` with a `jax.vmap`-batched bootstrap kernel; Section 2.6), but those paths do not support the headline table: GPU acceleration is a backend contract to be benchmarked on separate hardware, not evidence for a universal speed claim over **Stata** or **R**, whose ecosystems expose different acceleration models. The experimental Rayon-parallel Rust kernel and the `sp.iv(absorb=...)` HDFE path are likewise excluded because the submitted build remains reproducible without a compiled extension.

Module	Estimator	Reference	Max. N	sp	Ref.	Faster path
01_hdfe	HDFE 2-way FE	fixest	1,000,000	0.120	0.049	R 2.5 ×
02_csdiid	CS-DiD simple ATT	did	125,000	0.027	0.175	sp 6.5 ×
03_scm	Classical SCM	Synth	100	7.476	1.871	R 4.0 ×
04_dml	DML PLR (lin. learners)	DoubleML Python	10,000	0.008	0.009	tie 1.14×

Table 13: Track C measured performance on Apple Silicon arm64 / 8 cores / 24 GB, **StatsPAI** 1.30.1. Times are median seconds at the largest sample size, across 3 or 5 repetitions after one warmup, using seed-fixed DGPs. Bold denotes a gap of at least 1.5×. The DML row uses **DoubleML** Python with the same linear learners, five folds, and public-workflow construction cost. The Callaway–Sant’Anna row uses a homogeneous-effect DGP favourable to the vectorised group-time loop.

HDFE. A deterministic gravity-style panel with two fixed-effect factors (firm and year) and $N \in \{10^4, 10^5, 10^6\}$ rows compares `sp.fast.feols` with `fixest::feols` (Bergé 2018).

CS-DiD. A balanced staggered-adoption panel with $N_{\text{units}} \in \{1,000, 5,000, 25,000\}$ and five time periods compares `sp.callaway_santanna` with `did::att_gt` plus `did::aggte`.

SCM. A two-factor, 30-period synthetic-control panel with $n_{\text{donors}} \in \{20, 50, 100\}$ compares `sp.synth(method="classic")` with `Synth::synth`.

DML. A partially-linear DML design with linear nuisance learners and $N = 1,000, 5,000, \text{ or } 10,000$ compares `sp.dml(model="plr")` with **DoubleML** Python’s PLR estimator. Both construct their public workflow with the same **scikit-learn** linear learners and five folds inside the timed call. The design isolates wrapper and cross-fitting overhead rather than benchmarking random-forest training.

6.2. Measured performance results

Table 13 reports the largest-size row from each benchmark module, and Figure 3 reports the scaling curve over all measured sizes.

Reading the scaling figure. On the log-log axes of Figure 3, a roughly constant vertical offset between two parallel lines is a constant-factor runtime difference at matched asymptotic order, whereas a widening gap (diverging slopes) signals a difference in computational complexity. The figure shows whether each table gap is stable across the measured size range or, as flagged below for classical SCM, grows with problem size and therefore marks a genuine scaling target.

Headline findings. The results are mixed in the useful sense: they document where **StatsPAI** is competitive and where the reference implementation is still materially better.

- **HDFE 2-way FE:** at the largest measured row ($N = 10^6$), **fixest** is about 2.5× faster than `sp.fast.feols`. Both implementations remain sub-second at this size, so the practical conclusion is parity of usable scale rather than a **StatsPAI** speed win.

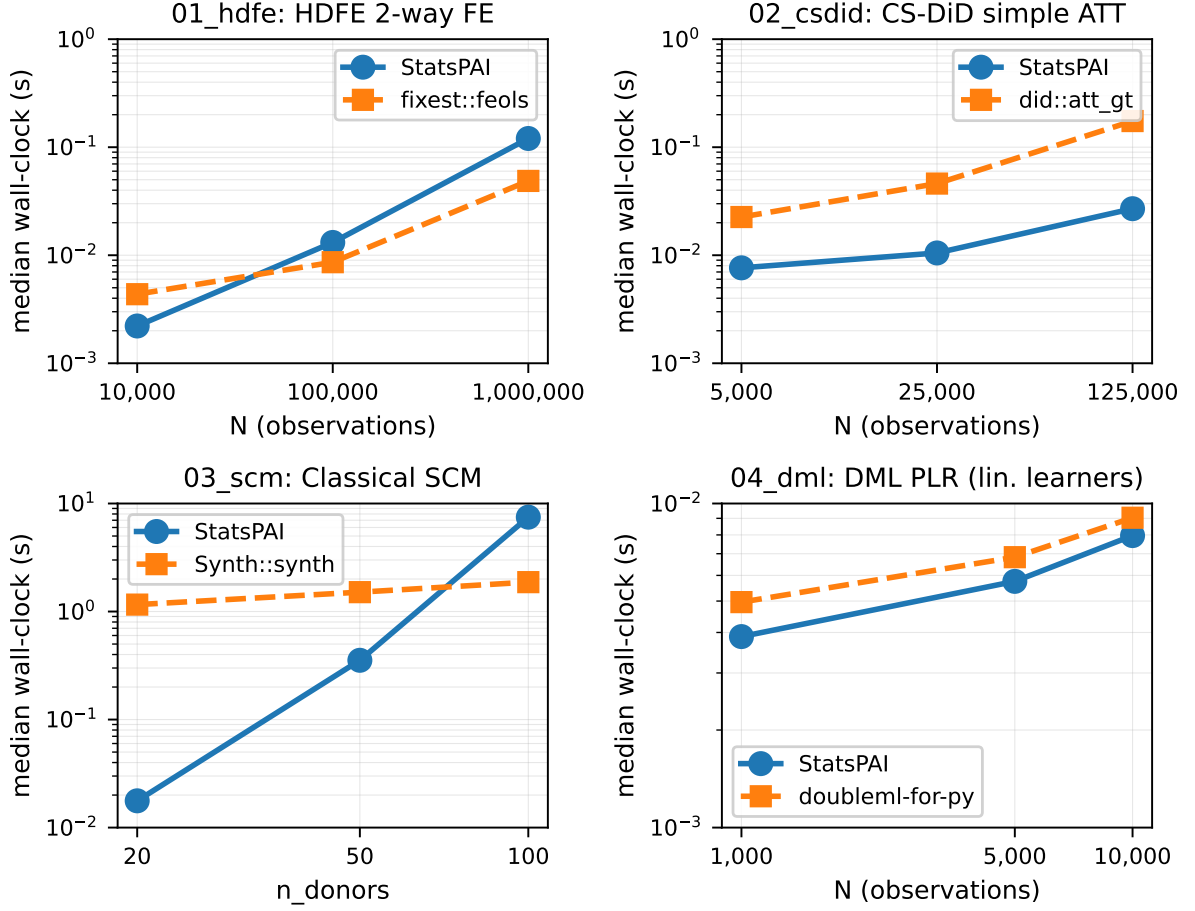


Figure 3: Log-log scaling of **StatsPAI** versus the declared reference for the four Track-C benchmarks. The harness runs both implementations on identical seed-fixed DGPs; points are median wall-clock seconds across 3 or 5 reps after a warmup. Hardware: Apple Silicon arm64 / 8 cores / 24 GB / macOS 26. The figure is regenerated by `tests/perf/compare_perf.py` on every benchmark run.

- **Callaway–Sant’Anna staggered DiD**: **StatsPAI** is about $6.5\times$ faster than **did** at 125,000 observations *on the homogeneous-effect DGP used here*. The homogeneous DGP is the cleanest case for the vectorized group-time-ATT loop; on a heterogeneous-effect DGP with many treated cohorts the gap should narrow because the IPW/DR cell-level computation dominates on both sides; a heterogeneous-DGP companion benchmark remains future work.
- **DML PLR with linear nuisance learners**: **StatsPAI** and **DoubleML** Python are effectively tied at the largest design (0.008 versus 0.009 seconds, ratio $1.14\times$). This same-language comparison supports only the narrow conclusion that the unified wrapper adds no material cost when nuisance fitting is cheap.
- **Classical SCM**: **Synth** is about $4.0\times$ faster than **StatsPAI** at $n_{\text{donors}} = 100$, while **StatsPAI** is the faster of the two at 20 and 50 donors (Figure 3), so the gap is one of scaling rather than fixed overhead. This is the main performance gap exposed by Track C. With more donors than pre-treatment periods the inner weight problem is rank-deficient, so each

candidate V of the nested optimization is solved by **scipy** SLSQP rather than the exact active-set solver used in the full-rank case; that inner loop is the next optimization target.

Interpretation. The benchmark supports a modest claim: the package’s unified surface does not impose a productivity-prohibitive overhead for representative workloads. One row favours **StatsPAI**, two favour the R reference at the largest size, and the same-language DML row is tied. It does not support a universal “Python is faster” claim. The SCM and HDFE rows are kept in the table precisely because they are useful negative evidence for future engineering priorities.

7. Agent-facing interface checks

The agent-facing design described in Section 3 is the most contestable claim in this paper: a reviewer may reasonably ask whether a machine-readable schema *actually* improves the behaviour of an LLM agent on a real empirical task, or whether a *statsmodels-plus-prompting* baseline performs equivalently well. That question needs frozen model releases, fixed prompts, matched tool budgets, and a scoring rubric; it cannot be inferred from schema coverage, and Section 7.3 states what answering it would take. The present section reports only the *mechanical* and *contractual* evidence that the agent-facing surface is correctly wired up. We refer to Patil, Zhang, Wang, and Gonzalez (2023) and Liang *et al.* (2023) as methodological precedents for tool-use and holistic-LLM benchmarks respectively.

7.1. Mechanical evidence

The function-schema layer is statically inspectable. We report in Table 14 both the function-level coverage (1,220 of 1,220 public functions emit a non-empty OpenAI-style JSON Schema; 416 currently expose curated or explicitly inherited agent metadata cards) and the parameter-level quality of those schemas (9,436 documented parameters across the 1,220 schemas, of which 35.5% carry an explicit default value and 5.0% carry enumerated choices). Parameter quality matters because the headline “1,220/1,220 emit a schema” figure is high by construction – the registry pipes *any* typed signature into a JSON Schema – whereas the stronger contract is that every parameter carries a non-empty description. We do not turn absence of package-distributed, package-wide OpenAI-style tool-schema catalogues in **statsmodels**, **linearmodels**, or **scikit-learn** into a numerical *zero* score: those projects target different interfaces, and Table 14 is an internal completeness audit, not a behavioural or package-quality ranking; they do not rule out external wrappers.

The same registry is exported as a versioned offline schema bundle covering tool schemas, function schemas, agent cards, result-payload validation, and provenance counts, so non-Python clients can discover the tool surface without importing **StatsPAI**.

To keep this surface honest as the package grows, **StatsPAI** ships a coverage ratchet for agent cards. The ratchet classifies registered functions into cumulative documentation tiers, records per-field and per-tier counters in a checked JSON floor, and fails CI on any regression. The tiers deliberately separate a minimal schema-ready description from richer assumptions, failure modes, alternatives, sample size guidance, and certified/validated evidence. Lowering the floor requires an explicit hand edit, so a schema-quality regression is visible in code review rather than hidden behind a large function count.

Audit item	Count	Missing	Share
Public functions/classes	1,220	—	100.0%
Emit non-empty OpenAI-style tool schema	1,220	0	100.0%
Have at least one typed parameter	1,188	—	—
Have non-empty function description	1,220	0	100.0%
Have curated or inherited agent metadata	416	804	34.1%
Parameters across all schemas	9,436	—	100.0%
with non-empty description	9,436	0	100.0%
with explicit default value	3,348	—	35.5%
with enumerated choices (<code>enum</code>)	468	—	5.0%

Table 14: Internal mechanical schema audit for **StatsPAI** 1.30.1. “Agent metadata” means at least one non-empty assumption, pre-condition, failure mode, limitation, or typical/minimum sample-size field after explicit inheritance. The remaining schemas still emit name, signature, and description fields, but are not counted as curated. Counts are regenerated from the live registry; this table makes no cross-package quality claim.

7.2. Contractual trace evidence

The agent-facing surface is exposed through an MCP server (Section 3.6) that negotiates Model Context Protocol revisions 2024-11-05, 2025-03-26, and 2025-06-18. The protocol test suite exercises the `initialize` handshake and version negotiation, `tools/list` and `tools/call` with structured content, and `isError` envelopes; the schema and registry suites check that registered functions emit tool schemas, that agent-card filters behave as documented, and that MCP tool discovery sees the same registry-backed surface as `sp.function_schema()`. These are the package’s own tests, not an external certification of protocol conformance.

For this submission we also execute one deterministic end-to-end trace through the live MCP server. The script `replication/scripts/ex07_agent_trace.py` writes a temporary `mpdta.csv` (the calibrated replica of Section 4.3, so the traced estimate -0.0330 is the replica value, not the original-data -0.03995), calls `tools/list`, fits `callaway_santanna` with `as_handle=true`, sends the returned handle to `audit_result`, and then sends the same handle to `honest_did_from_result`. It finally resolves the returned citation keys through the MCP `bibtex` tool against the curated `paper.bib`, so the trace exercises the citation anti-hallucination contract rather than merely reporting key strings. The same trace also sends a stale result-handle call to `audit_result` and checks that the MCP response is a recoverable `isError` envelope with an actionable `refit` hint, so failure recovery is exercised rather than only described. Table 15 summarizes the trace; the normalized transcript is included in the replication archive. The trace verifies the agent-specific claim that the estimate $\rightarrow$ audit $\rightarrow$ sensitivity chain is executable without ferrying arrays or hand-copying citations through the model context.

7.3. No behavioural benchmark claim

Mechanical and contractual evidence does not answer whether an LLM agent using this interface produces *better empirical analyses* than one using **statsmodels** or R packages through a Jupyter MCP shim. That question requires frozen model releases, fixed prompts, matched tool budgets, and a scoring rubric, and belongs in a separate behavioural study. To

Trace item	Value
MCP tools listed	559
Estimate	-0.0330 (SE 0.0078)
Audit checks	4
Honest-DiD max rejecting M	0.0074
Recoverable stale-handle error	isError=True; hint present
Verified citation keys	callaway2021difference, rambachan2023more
BibTeX entries resolved	2

Table 15: Deterministic MCP trace used for the agent-facing interface check, including one recoverable stale-handle error path. Ephemeral result handles are normalized in the transcript shipped with the replication archive (`replication/results/ex07_agent_trace.txt`).

make that deferral auditable rather than vague, the archive includes `replication/results/agent_benchmark_protocol.md`, which records the matched arms, task families, scoring dimensions, leakage controls, and minimum report items required before any behavioural claim is made. The JSS contribution here is contractual: schemas, typed errors, result handles, citations, and the deterministic trace are reproducible interfaces, regenerated from the same validation ledger that governs human-facing calls rather than from a separate prompt-engineering layer, and they keep later benchmark scores from being confounded with missing metadata or unverifiable citations.

8. Computational details

StatsPAI installs from PyPI without compiled extensions of its own; its numerical core runs on the scientific Python stack (**numba**’s JIT compiler is the only compilation step, and it is optional at runtime). The submitted artifact uses Python 3.12.1, **NumPy** 1.26.4, **SciPy** 1.13.0, **pandas** 2.2.0, **statsmodels** 0.14.1, **scikit-learn** 1.4.0, **linearmodels** 6.0, and **matplotlib** 3.8 (Hunter 2007). Optional heavy backends are isolated under extras and are lazily imported. The submitted numerical and timing claims do not require a compiled Rust extension, GPU, or commercial software.

The R parity harness is pinned by `tests/r_parity/renv.lock`. The reference environment uses R 4.5.2; the packages behind the twelve-estimator suite are **fixest** 0.14.0, **did** 2.3.0, **Synth** 1.1-10, **synthdid** 0.0.9, **rdrobust** 3.0.0, **rddensity** 2.6, **HonestDiD** 0.2.8, **DoubleML** 1.0.2, **grf** 2.6.1, and **MatchIt** 4.7.2, joined by **fact** 2.4.1 and **interflex** 1.4.0 for the three-ecosystem worked example; the complete dependency closure for all 89 modules (including **augsynth**, **gsynth**, **lme4**, **CBPS** (Imai and Ratkovic 2014), **ebal** (Hainmueller 2012), **optmatch** (Hansen and Klopfer 2006), **Matching** (Sekhon 2011), and **sbw** (Wang and Zubizarreta 2020)) is recorded with exact versions and GitHub SHAs in the lockfile and per-row JSON provenance. The Stata bridge uses Stata 18 MP and records official/community command versions in `tests/stata_parity/STATA_ENVIRONMENT.md`. All 85 R-joined Stata modules and all 89 R modules reproduce their frozen JSON outputs with zero drift; the Stata leg requires a licence, so the archive ships the frozen JSONs, matching do-files, and engine provenance, and the main paper remains auditable from the Python/R artifacts.

8.1. Distribution and installation

The package is distributed through PyPI (`pip install statspai`); this article describes release 1.30.1 (tag `v1.30.1`). Optional extras are exposed as `dev`, `performance`, `bayes`, `deepiv`, `neural`, `fixest`, `rd-cct`, and `plotting` bundles. The `rd-cct` extra installs the official **rdrobust** Python port (`rdrobust` ≥ 2.0 , maintained by the **rdrobust** authors), which backs the optional `bwselect="cct"` path of Section 4.4; that port is GPL-3 licensed, which is why it is an opt-in extra installed by the user rather than a bundled dependency of the MIT-licensed core. No parity claim in this paper depends on it: the native default selector reproduces its numbers on the audited fixture without it.

The test suite runs on Linux, macOS, and Windows in continuous integration; the acceptance evidence for this paper is the Track A–D artifacts, worked examples, and audit scripts, each regenerated rather than quoted.

8.2. Reproducibility of this paper

Reviewers can rebuild the paper’s evidence along three paths, driven by one standalone script, `Paper-JSS/replication/reproduce.py`. The *licence-free path* (`-tier 1`; the default) needs only the pinned Python environment and runs in under ten minutes on commodity hardware, inside the JSS one-hour threshold for a reviewer verification path. It is important to be precise about what it does. It *recomputes* the seven worked examples, every manuscript listing and transcript, the known-truth recovery tests, the seed-replicated forest analysis, and the configuration map; it only *re-derives tables* from frozen artifacts for the three experiments that cost hours — Track A parity, Track B Monte Carlo, and Track C timing. Each of those has a separate, documented recomputation entry point (Table 16). The *R path* (`-tier 2`) adds the pinned `renv` environment and re-executes the cross-language parity modules themselves, which the licence-free path can only replay from frozen artifacts; by construction the paper’s parity claim is a comparison against R, so auditing the rows at full strength requires this path. The *Stata path* (`-tier 3`) re-executes the migration bridges and requires a Stata licence. A Dockerfile reproduces the licence-free environment exactly.

Two further checks are on the paper rather than the package. The listing audit replays each printed transcript statement by statement and compares the printed output line for line with what the code produces. And both sides of every parity row are re-derivable: `tests/r_parity/verify_reproduce.py` re-runs each R reference on the committed bytes, and `tests/r_parity/verify_reproduce_py.py` does the same for the **StatsPAI** side and also regenerates each fixture CSV byte for byte; all 89 modules clear a 10^{-9} reproduction floor with no exemptions. On the licence-free path the parity, coverage, and timing experiments are frozen evidence on that path; Table 16 lists how to recompute each.

The replication archive contains the manuscript sources, replication scripts, selected package source, validation tests, R/Stata/reference parity artifacts, Monte Carlo and performance artifacts, and the deterministic MCP trace, and fits within the JSS attachment limit.

9. Discussion

StatsPAI is a unification and validation project, not a new-estimator paper. The claim is that a single Python package can expose the applied causal workflow through shared result

Artifact	Tier 1 does	Full recomputation	Needs	Time
Worked examples, listings (Sec. 4)	recomputes	<code>reproduce.py</code> (Tier 1)	Python	<10 min
Track A, StatsPAI side (Tab. 17)	re-reads JSON	<code>-recompute parity-py</code> (and fixture bytes)	Python	~15 min
Track A, R side	re-reads JSON	<code>-tier 2</code>	R 4.5.2, <code>renv.lock</code>	30–60 min
Track A, Stata side	re-reads JSON	<code>-tier 3</code>	Stata 18	~30 min
Forest seeds (Tab. 10)	re-analyses frozen draws	<code>-recompute forest-seed</code>	Python; R for grf side	~30 min + ~20 min
Track B (Tabs. 11–12)	re-reads JSON	<code>-recompute montecarlo</code>	Python	~80 min
Track C (Fig. 3)	re-reads JSON	<code>-recompute performance</code>	Python; R for reference legs	hardware-dependent

Table 16: What the licence-free path recomputes and how to recompute the rest (options of `replication/reproduce.py`); each writes to the artifact path the paper reads.

surfaces, explicit validation status, reproducible parity artifacts, and machine-readable agent schemas. It is not a claim that **StatsPAI** dominates specialized reference packages in their home domains, nor that the MCP interface replaces empirical judgement.

The main limitations are as follows. First, 568 stable auto-registered symbols remain API-stable but not parity-backed (292 class-like, 276 function-like), and the other 93 symbols of the 661-symbol `api_stable` tier carry API, unit-contract, convention, or supplemental evidence only. The validated claim is limited to certified/validated entries and to the named rows and modules in the validation-suite evidence; the validation-evidence audit lists nine scoped limitation rows, each tied to a named function, evidence grade, and unsupported variant. Second, only one Track A row remains a methodological/T4 disclosure: classical SCM on the calibrated Basque replica, where reference implementations choose measurably different local optima; on the original `Synth::basque` bytes the same specification brings all three implementations within 4×10^{-4} (Section 5.4), so the disclosure is about data-dependent non-uniqueness, not a standing numerical defect. The causal-forest row is T3: the two forest engines are equivalent to within a tenth of a sampling standard error and equally noisy across seeds, which is an equivalence claim, not parity. The 3 Track A rows served by third-party Python libraries validate **StatsPAI**’s wrappers of those libraries, not native algorithms. The configuration map behind `sp.validation_scope` covers the twelve suite estimators; for the rest of the registry the evidence notes name the artifact but not its configuration. Third, the **fect** and **interflex** cross-validation selectors for the factor number, penalty, and kernel bandwidth are ported in 1.29.0 but can only be graded T3: their folds are random, so the three-ecosystem parity of Section 4.7 is established at user-specified tuning values, and the selectors are checked by given-fold score equality plus across-seed agreement of the selection. Fourth, Stata reproduction requires a separate licence, so Stata rows are migration bridges rather than the licence-free core of the archive; 4 of the 89 modules carry no Stata reference, each with a measured reason. Fifth, Bayesian, causal-discovery, and behavioural-agent evaluations require different loss functions and are outside this software-validation article. Sixth, within DiD, **StatsPAI** implements the apparatus of the Baker *et al.* (2026) practitioner’s guide but not the efficient combination of comparison groups and baseline periods (Chen, Sant’Anna, and Xie 2025) or selection-based pre-trend diagnostics (Ghanem, Sant’Anna, and Wüthrich 2026).

The unification itself has costs worth stating plainly, because they are the price of the design rather than defects of it. A unified API necessarily lags its upstream references: when **did**

or **fixest** ships a new option, **StatsPAI** users wait for the ported, parity-tested version, and the validation regime that makes the port trustworthy is exactly what makes it slower to ship. The registry’s own census quantifies a second cost: 568 of the registered symbols carry API-contract evidence only, so a user who strays from the validated core onto a long-tail helper is relying on software-engineering guarantees, not statistical validation – which is why the tier is printed at call time rather than in an appendix. Third, the evidence regime itself has a maintenance bill: every correctness fix in Table 18 exists because a parity or coverage artifact flagged it, and keeping 89 modules pinned against two external ecosystems is ongoing work that a single-method package does not carry.

A fourth cost is a limit of the regime rather than of the package. A parity ledger validates the paths it exercises and is silent about the rest: the 1.25.0 entry of Table 18, a continuous-treatment forest path no Track A or Track B row reaches, surfaced only in a worked example. That is why the registry records *which* artifact backs a label, why `sp.validation_scope` reports the configuration an artifact exercised, and why the worked examples are executed end-to-end.

The roadmap is to resolve or further bound the remaining classical-SCM reference-disagreement disclosure, extend $B = 1,000$ coverage beyond the headline rows, close the two measured performance gaps of Section 6, and move the agent interface from mechanical conformance to scored behavioural benchmarking.

A final limitation is institutional: a large surface is maintained by a small team with one principal author. The risk is bounded by properties verifiable from the archive: the package is MIT-licensed with no relicensing clause, so the validated core can be forked; every output-changing fix is pinned by a regression test; and the parity harness is scripted and environment-pinned (`renv.lock`, recorded Stata versions, per-row JSON provenance), so any third party with the reference runtimes can re-execute it.

Author contributions

B. W. designed and implemented **StatsPAI**, the validation harnesses and registry API, and drafted the manuscript. S. R. supervised the applied-econometrics programme, advised on scope and validation priorities, and reviewed the manuscript. Both authors approved the final version.

Acknowledgements

The authors thank Stanford REAP for institutional support and feedback, and the maintainers of the Python, R, and Stata packages used as parity references. Any remaining errors are the authors’ own.

Conflict of interest

StatsPAI is MIT-licensed open-source software. The authors are affiliated with StatsPAI Inc., which maintains the package; CoPaper.AI is a commercial downstream product that may call **StatsPAI** but is not required to use it. B. W. is the creator and principal maintainer. No

external funder had a role in the validation studies, analysis, or manuscript preparation.

AI usage disclosure

Portions of code documentation and manuscript text were prepared with large-language-model assistance. The authors selected the claims, reviewed the text, and remain responsible for all statistical decisions. Numerical claims are tied to tests, replication scripts, frozen JSON artifacts, and cited references; citations are checked by the verifier in Section 3.5.

References

- Abadie A, Diamond A, Hainmueller J (2010). “Synthetic control methods for comparative case studies: Estimating the effect of California’s tobacco control program.” *Journal of the American Statistical Association*, **105**(490), 493–505. doi:10.1198/jasa.2009.ap08746.
- Abadie A, Diamond A, Hainmueller J (2011). “Synth: An R Package for Synthetic Control Methods in Comparative Case Studies.” *Journal of Statistical Software*, **42**(13), 1–17. doi:10.18637/jss.v042.i13.
- Abadie A, Gardeazabal J (2003). “The Economic Costs of Conflict: A Case Study of the Basque Country.” *American Economic Review*, **93**(1), 113–132. doi:10.1257/000282803321455188.
- Andrews I, Roth J, Pakes A (2023). “Inference for Linear Conditional Moment Inequalities.” *Review of Economic Studies*, **90**(6), 2763–2791. doi:10.1093/restud/rdad004.
- Angrist JD, Pischke JS (2009). *Mostly Harmless Econometrics: An Empiricist’s Companion*. Princeton University Press. ISBN 978-0-691-12035-5. doi:10.1515/9781400829828.
- Anthropic (2024). “Model Context Protocol Specification.” <https://modelcontextprotocol.io/>. Accessed 2026-09-04.
- Arkhangelsky D, Athey S, Hirshberg DA, Imbens GW, Wager S (2021). “Synthetic difference-in-differences.” *American Economic Review*, **111**(12), 4088–4118. doi:10.1257/aer.20190159.
- Athey S, Tibshirani J, Wager S (2019). “Generalized random forests.” *The Annals of Statistics*, **47**(2), 1148–1178. doi:10.1214/18-aos1709.
- Bach P, Chernozhukov V, Kurz MS, Spindler M (2022). “DoubleML – An Object-Oriented Implementation of Double Machine Learning in Python.” *Journal of Machine Learning Research*, **23**(53), 1–6. URL <https://jmlr.org/papers/v23/21-0862.html>.
- Bach P, Kurz MS, Chernozhukov V, Spindler M, Klaassen S (2024). “DoubleML: An Object-Oriented Implementation of Double Machine Learning in R.” *Journal of Statistical Software*, **108**(3), 1–56. doi:10.18637/jss.v108.i03.
- Baker A, Callaway B, Cunningham S, Goodman-Bacon A, Sant’Anna PHC (2026). “Difference-in-Differences Designs: A Practitioner’s Guide.” *Journal of Economic Literature*, **64**(2), 498–557. doi:10.1257/jel.20251650.

- Battocchi K, Dillon E, Hei M, Lewis G, Oka P, Oprescu M, Syrgkanis V (2019). “EconML: A Python Package for ML-Based Heterogeneous Treatment Effects Estimation.” URL <https://github.com/py-why/EconML>.
- Ben-Michael E, Feller A, Rothstein J (2021). “The Augmented Synthetic Control Method.” *Journal of the American Statistical Association*, **116**(536), 1789–1803. doi:10.1080/01621459.2021.1929245.
- Bergé L (2018). “Efficient estimation of maximum likelihood models with multiple fixed-effects: the R package FENmlm.” *Technical Report 18-13*, DEM Discussion Paper, Department of Economics, University of Luxembourg.
- Blackwell M (2014). “A Selection Bias Approach to Sensitivity Analysis for Causal Effects.” *Political Analysis*, **22**(2), 169–182. doi:10.1093/pan/mpt006.
- Borusyak K, Jaravel X, Spiess J (2024). “Revisiting event-study designs: Robust and efficient estimation.” *Review of Economic Studies*, **91**(6), 3253–3285. doi:10.1093/restud/rdae007.
- Brodersen KH, Gallusser F, Koehler J, Remy N, Scott SL (2015). “Inferring causal impact using Bayesian structural time-series models.” *The Annals of Applied Statistics*, **9**(1), 247–274. doi:10.1214/14-aos788.
- Callaway B, Sant’Anna PHC (2021). “Difference-in-differences with multiple time periods.” *Journal of Econometrics*, **225**(2), 200–230. doi:10.1016/j.jeconom.2020.12.001.
- Calonico S, Cattaneo MD, Titiunik R (2014). “Robust nonparametric confidence intervals for regression-discontinuity designs.” *Econometrica*, **82**(6), 2295–2326. doi:10.3982/ECTA11757.
- Calonico S, Cattaneo MD, Titiunik R (2015). “rdrubust: An R Package for Robust Non-parametric Inference in Regression-Discontinuity Designs.” *The R Journal*, **7**(1), 38–51. doi:10.32614/RJ-2015-004.
- Cameron AC, Gelbach JB, Miller DL (2008). “Bootstrap-Based Improvements for Inference with Clustered Errors.” *Review of Economics and Statistics*, **90**(3), 414–427. doi:10.1162/rest.90.3.414.
- Card D (1995). “Using Geographic Variation in College Proximity to Estimate the Return to Schooling.” In LN Christofides, EK Grant, R Swidinsky (eds.), *Aspects of Labour Market Behaviour: Essays in Honour of John Vanderkamp*, pp. 201–222. University of Toronto Press, Toronto. ISBN 0802028640.
- Cattaneo MD, Jansson M, Ma X (2018). “Manipulation Testing Based on Density Discontinuity.” *The Stata Journal*, **18**(1), 234–261. doi:10.1177/1536867X1801800115.
- Cattaneo MD, Jansson M, Ma X (2020). “Simple Local Polynomial Density Estimators.” *Journal of the American Statistical Association*, **115**(531), 1449–1455. doi:10.1080/01621459.2019.1635480.
- Chen H, Harinen T, Lee JY, Yung M, Zhao Z (2020). “CausalML: Python Package for Causal Machine Learning.” doi:10.48550/arXiv.2002.11631.

- Chen X, Sant’Anna PHC, Xie H (2025). “Efficient Difference-in-Differences and Event Study Estimators.” ArXiv preprint, first posted 2025-06-21, [2506.17729](#).
- Chernozhukov V, Chetverikov D, Demirer M, Duflo E, Hansen C, Newey W, Robins J (2018). “Double/debiased machine learning for treatment and structural parameters.” *The Econometrics Journal*, **21**(1), C1–C68. [doi:10.1111/ectj.12097](#).
- Clarke D, Paila  ir D, Athey S, Imbens G (2024). “On synthetic difference-in-differences and related estimation methods in Stata.” *The Stata Journal*, **24**(4), 557–598. [doi:10.1177/1536867X241297914](#).
- Conley TG (1999). “GMM estimation with cross sectional dependence.” *Journal of Econometrics*, **92**(1), 1–45. [doi:10.1016/S0304-4076\(98\)00084-0](#).
- Correia S, Guimar  es P, Zylkin T (2020). “Fast Poisson estimation with high-dimensional fixed effects.” *Stata Journal*, **20**(1), 95–115. [doi:10.1177/1536867X20909691](#).
- Dehejia RH, Wahba S (1999). “Causal Effects in Nonexperimental Studies: Reevaluating the Evaluation of Training Programs.” *Journal of the American Statistical Association*, **94**(448), 1053–1062. [doi:10.1080/01621459.1999.10473858](#).
- Ghanem D, Sant’Anna PHC, W  thrich K (2026). “Selection and Parallel Trends.” ArXiv preprint, v15 dated 2026-07-24 (first posted 2022-03-17), [2203.09001](#).
- Goodman-Bacon A (2021). “Difference-in-differences with variation in treatment timing.” *Journal of Econometrics*, **225**(2), 254–277. [doi:10.1016/j.jeconom.2021.03.014](#).
- Hainmueller J (2012). “Entropy Balancing for Causal Effects: A Multivariate Reweighting Method to Produce Balanced Samples in Observational Studies.” *Political Analysis*, **20**(1), 25–46. [doi:10.1093/pan/mpr025](#).
- Hainmueller J, Mummolo J, Xu Y (2019). “How Much Should We Trust Estimates from Multiplicative Interaction Models? Simple Tools to Improve Empirical Practice.” *Political Analysis*, **27**(2), 163–192. [doi:10.1017/pan.2018.46](#).
- Hansen BB, Klopfer SO (2006). “Optimal Full Matching and Related Designs via Network Flows.” *Journal of Computational and Graphical Statistics*, **15**(3), 609–627. [doi:10.1198/106186006X137047](#).
- Harris CR, Millman KJ, van der Walt SJ, Gommers R, Virtanen P, Cournapeau D, Wieser E, Taylor J, *et al.* (2020). “Array Programming with NumPy.” *Nature*, **585**(7825), 357–362. [doi:10.1038/s41586-020-2649-2](#).
- Ho DE, Imai K, King G, Stuart EA (2011). “MatchIt: Nonparametric Preprocessing for Parametric Causal Inference.” *Journal of Statistical Software*, **42**(8), 1–28. [doi:10.18637/jss.v042.i08](#).
- Hunter JD (2007). “Matplotlib: A 2D Graphics Environment.” *Computing in Science & Engineering*, **9**(3), 90–95. [doi:10.1109/MCSE.2007.55](#).
- Imai K, Ratkovic M (2014). “Covariate Balancing Propensity Score.” *Journal of the Royal Statistical Society Series B*, **76**(1), 243–263. [doi:10.1111/rssb.12027](#). Fong, Ratkovic & Imai are the CBPS R package maintainers (<https://github.com/kosukeimai/CBPS>).

- Imbens GW, Wooldridge JM (2009). “Recent Developments in the Econometrics of Program Evaluation.” *Journal of Economic Literature*, **47**(1), 5–86. doi:[10.1257/jel.47.1.5](https://doi.org/10.1257/jel.47.1.5).
- Kleibers C, Zeileis A (2008). *Applied Econometrics with R*. Springer, New York. doi:[10.1007/978-0-387-77318-6](https://doi.org/10.1007/978-0-387-77318-6).
- LaLonde RJ (1986). “Evaluating the Econometric Evaluations of Training Programs with Experimental Data.” *American Economic Review*, **76**(4), 604–620.
- Lam SK, Pitrou A, Seibert S (2015). “Numba: A LLVM-Based Python JIT Compiler.” In *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, pp. 1–6. doi:[10.1145/2833157.2833162](https://doi.org/10.1145/2833157.2833162).
- Lee DS (2008). “Randomized experiments from non-random selection in U.S. House elections.” *Journal of Econometrics*, **142**(2), 675–697. doi:[10.1016/j.jeconom.2007.05.004](https://doi.org/10.1016/j.jeconom.2007.05.004).
- Li Z, Strezhnev A (2025). “Benchmarking Parallel Trends Violations in Regression Imputation Difference-in-Differences.” doi:[10.31235/osf.io/ngr3d_v1](https://doi.org/10.31235/osf.io/ngr3d_v1). SocArXiv preprint, v1 posted 2025-07-11.
- Liang P, Bommasani R, Lee T, Tsipras D, Soylu D, Yasunaga M, Zhang Y, Narayanan D, Wu Y, Kumar A, Newman B, Yuan B, Yan B, Zhang C, Cosgrove C, Manning CD, Ré C, Acosta-Navas D, Hudson DA, Zelikman E, Durmus E, Ladhak F, Rong F, Ren H, Yao H, Wang J, Santhanam K, Orr L, Zheng L, Yuksekgonul M, Suzgun M, Kim N, Guha N, Chatterji N, Khattab O, Henderson P, Huang Q, Chi R, Xie SM, Santurkar S, Ganguli S, Hashimoto T, Icard T, Zhang T, Chaudhary V, Wang W, Li X, Mai Y, Zhang Y, Koreeda Y (2023). “Holistic Evaluation of Language Models.” *Transactions on Machine Learning Research*. [2211.09110](https://arxiv.org/abs/2211.09110).
- Liu L, Wang Y, Xu Y (2024). “A Practical Guide to Counterfactual Estimators for Causal Inference with Time-Series Cross-Sectional Data.” *American Journal of Political Science*, **68**(1), 160–176. doi:[10.1111/ajps.12723](https://doi.org/10.1111/ajps.12723).
- McKinney W (2010). “Data Structures for Statistical Computing in Python.” In *Proceedings of the 9th Python in Science Conference*, pp. 56–61. doi:[10.25080/Majora-92bf1922-00a](https://doi.org/10.25080/Majora-92bf1922-00a).
- Mou H, Liu L, Xu Y (2023). “Panel Data Visualization in R (panelView) and Stata (panelview).” *Journal of Statistical Software*, **107**(7), 1–20. doi:[10.18637/jss.v107.i07](https://doi.org/10.18637/jss.v107.i07).
- Patil SG, Zhang T, Wang X, Gonzalez JE (2023). “Gorilla: Large Language Model Connected with Massive APIs.” ArXiv preprint, first posted 2023-05-24, [2305.15334](https://arxiv.org/abs/2305.15334).
- Pedregosa F, Varoquaux G, Gramfort A, Michel V, Thirion B, Grisel O, Blondel M, Prettenhofer P, Weiss R, Dubourg V, Vanderplas J, Passos A, Cournapeau D, Brucher M, Perrot M, Duchesnay É (2011). “Scikit-learn: Machine Learning in Python.” *Journal of Machine Learning Research*, **12**, 2825–2830. URL <https://jmlr.org/papers/v12/pedregosa11a.html>.
- Pustejovsky JE, Tipton E (2018). “Small-Sample Methods for Cluster-Robust Variance Estimation and Hypothesis Testing in Fixed Effects Models.” *Journal of Business & Economic Statistics*, **36**(4), 672–683. doi:[10.1080/07350015.2016.1247004](https://doi.org/10.1080/07350015.2016.1247004).

- Rambachan A, Roth J (2023). “A more credible approach to parallel trends.” *Review of Economic Studies*, **90**(5), 2555–2591. doi:10.1093/restud/rdad018.
- Rios-Avila F, Sant’Anna PHC, Callaway B, Naqvi A (2022). “CSDID: Stata Module for the Estimation of Difference-in-Differences Models with Multiple Time Periods.” https://github.com/friosavila/csdid_drdid. Stata user-contributed package (GitHub); accessed 2026-09-04.
- Roodman D, Nielsen MØ, MacKinnon JG, Webb MD (2019). “Fast and wild: Bootstrap inference in Stata using boottest.” *The Stata Journal*, **19**(1), 4–60. doi:10.1177/1536867x19830877.
- Roth J (2026). “Interpreting event-studies from recent difference-in-differences methods.” *The Japanese Economic Review*, **77**(2), 275–288. doi:10.1007/s42973-026-00235-x. ArXiv:2401.12309.
- Roth J, Sant’Anna PHC, Bilinski A, Poe J (2023). “What’s trending in difference-in-differences? A synthesis of the recent econometrics literature.” *Journal of Econometrics*, **235**(2), 2218–2244. doi:10.1016/j.jeconom.2023.03.008.
- Sant’Anna PHC, Zhao J (2020). “Doubly Robust Difference-in-Differences Estimators.” *Journal of Econometrics*, **219**(1), 101–122. doi:10.1016/j.jeconom.2020.06.003.
- Seabold S, Perktold J (2010). “Statsmodels: Econometric and Statistical Modeling with Python.” In *Proceedings of the 9th Python in Science Conference*, pp. 92–96. doi:10.25080/Majora-92bf1922-011.
- Sekhon JS (2011). “Multivariate and Propensity Score Matching Software with Automated Balance Optimization: The Matching Package for R.” *Journal of Statistical Software*, **42**(7), 1–52. doi:10.18637/jss.v042.i07.
- Sharma A, Kiciman E (2020). “DoWhy: An End-to-End Library for Causal Inference.” doi:10.48550/arXiv.2011.04216.
- Staiger D, Stock JH (1997). “Instrumental Variables Regression with Weak Instruments.” *Econometrica*. doi:10.2307/2171753.
- Stock JH, Yogo M (2005). “Testing for Weak Instruments in Linear IV Regression.” *Identification and Inference for Econometric Models*. doi:10.1017/cbo9780511614491.006.
- Sun L, Abraham S (2021). “Estimating dynamic treatment effects in event studies with heterogeneous treatment effects.” *Journal of Econometrics*, **225**(2), 175–199. doi:10.1016/j.jeconom.2020.09.006.
- Virtanen P, Gommers R, Oliphant TE, Haberland M, Reddy T, Cournapeau D, Burovski E, Peterson P, *et al.* (2020). “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python.” *Nature Methods*, **17**(3), 261–272. doi:10.1038/s41592-019-0686-2.
- Wager S, Athey S (2018). “Estimation and inference of heterogeneous treatment effects using random forests.” *Journal of the American Statistical Association*, **113**(523), 1228–1242. doi:10.1080/01621459.2017.1319839.

- Wang B, Rozelle S (2026). “StatsPAI: A Unified, Agent-Native Python Toolkit for Causal Inference and Applied Econometrics.” *Journal of Open Source Software*, **11**(125), 10604. doi:10.21105/joss.10604. URL <https://doi.org/10.21105/joss.10604>.
- Wang Y, Zubizarreta JR (2020). “Minimal Dispersion Approximately Balancing Weights: Asymptotic Properties and Practical Considerations.” *Biometrika*, **107**(1), 93–105. doi:10.1093/biomet/asz050.
- Wing C, Yozwiak M, Hollingsworth A, Freedman S, Simon KI (2024). “Designing Difference-in-Difference Studies with Staggered Treatment Adoption: Key Concepts and Practical Guidelines.” *Annual Review of Public Health*, **45**, 485–505. doi:10.1146/annurev-publhealth-061022-050825.
- Xu Y (2017). “Generalized synthetic control method: Causal inference with interactive fixed effects models.” *Political Analysis*, **25**(1), 57–76. doi:10.1017/pan.2016.2.
- Zeileis A, Köll S, Graham N (2020). “Various Versatile Variances: An Object-Oriented Implementation of Clustered Covariances in R.” *Journal of Statistical Software*, **95**(1), 1–36. doi:10.18637/jss.v095.i01.

A. The complete Track A ledger

Table 8 in Section 5.3 prints a twelve-row slice of Track A, chosen to span the validation suite. The slice is a reading aid, not the evidence. The evidence is Table 17 below: every one of the 89 modules, with the worst residual gap each one carries against its canonical R reference and, for 85 of them, against a canonical or audited Stata bridge.

The difference columns are worst-case over each module’s headline rows and cover *point estimates only*; every SE row of every passing module is gated separately (Section 5.8). The parenthetical notes name the convention, delegation, or common specification under which a row is claimed, and italic text in the Stata column records a *measured* reason that no portable Stata artifact exists.

ID	Method	Worst diff vs R	Worst diff vs Stata	Verdict
01	OLS + HC1 SE	$\text{rel} \leq 1.1 \times 10^{-12}$	$\text{rel} \leq 1.3 \times 10^{-12}$	PASS
02	2SLS + HC1 SE	$\text{rel} \leq 1.1 \times 10^{-11}$	$\text{rel} \leq 1.1 \times 10^{-11}$	PASS
03	HDFE 2-way FE	$\text{rel} \leq 5.2 \times 10^{-15}$ (fixest/reghdfe small-sample correction)	$\text{rel} \leq 2.9 \times 10^{-15}$	PASS
04	CS-DiD simple ATT	$\text{rel} \leq 1.3 \times 10^{-15}$ (estimate and SE both $\text{rel} < 1 \times 10^{-9}$)	$\text{rel} \leq 1.3 \times 10^{-15}$	PASS
05	Sun–Abraham event study	$\text{rel} \leq 2.8 \times 10^{-11}$ (fixest agg='att' summary parity)	$\text{rel} \leq 2.7 \times 10^{-11}$	PASS
06	RD CCT bias-corrected	$\text{rel} \leq 5.7 \times 10^{-14}$ (native CCT default- <i>h</i> ; official Py port recorded separately)	$\text{rel} \leq 9.4 \times 10^{-11}$	PASS
07	Classical SCM	$\text{rel} \leq 0.024$ (T4 reference disagreement: native tracks Stata; R Synth differs; exact recovery on identified DGP 52)	$\text{rel} \leq 0.00042$	<i>GAP</i>

(continued on the next page)

(Table 17 continued)

ID	Method	Worst diff vs R	Worst diff vs Stata	Verdict
08	DML PLR (LinReg learners)	$\text{rel} \leq 0$ (shared explicit fold_id)	$\text{rel} \leq 3.7 \times 10^{-15}$	PASS
09	RD density (CJM)	$\text{rel} \leq 9.3 \times 10^{-12}$ (native CJM/rddensity default parity)	$\text{rel} \leq 1.8 \times 10^{-11}$	PASS
10	Honest DiD bounds	$\text{abs} \leq 1.41 \times 10^{-8}$ (native FLCI; vs HonestDiD with exact quantile, and closed form at M=0; shipped HonestDiD's simulated quantile shown separately)	$\text{abs} \leq 9.38 \times 10^{-9}$	PASS
11	PSM 1:1 NN	$\text{rel} \leq 1.2 \times 10^{-15}$ (se_teffects_ai joins Stata teffects psmatch's Abadie-Imbens 2016 estimated-score variance at $\text{rel} 7 \times 10^{-8}$ via the abadie_imbens_2016 SE option; the default psmatch2 ai(1) SE is a documented convention (sample vs population ATT, no score term))	$\text{rel} \leq 2 \times 10^{-16}$	PASS
12	Synthetic DID	$\text{rel} \leq 7.8 \times 10^{-16}$ (native Frank-Wolfe/zeta ATT parity; backend-native placebo SE diagnostics)	$\text{rel} \leq 7.2 \times 10^{-8}$	PASS
13	Causal forest (AIPW)	$\text{rel} \leq 0.00026$ (T3; single draw per engine, graded by seed-replicated equivalence within 0.1 sampling SE)	<i>bridge artifact not materialized: Stata 19's official cate command is the candidate causal-forest/AIPW reference, but the verified local runtime is Stata 18 and 'which cate' fails.</i>	PASS
14	OLS + cluster-robust SE	$\text{rel} \leq 3 \times 10^{-14}$	$\text{rel} \leq 2.2 \times 10^{-14}$	PASS
15	HDFE + cluster SE	$\text{rel} \leq 5.2 \times 10^{-15}$ (fixest/reghdfe nested-FE cluster correction)	$\text{rel} \leq 2.9 \times 10^{-15}$	PASS
16	BJS imputation	$\text{rel} \leq 4.8 \times 10^{-8}$ (R/Stata simple-ATT point parity; SE rows side-specific)	$\text{rel} \leq 3.5 \times 10^{-7}$	PASS
17	Wooldridge ETWFE	$\text{rel} \leq 1.8 \times 10^{-13}$ (emfx aggregation and cluster SE matched)	$\text{rel} \leq 3.9 \times 10^{-14}$	PASS
18	Augmented SCM	$\text{rel} \leq 7.9 \times 10^{-6}$ (native centered Ridge+SCM parity; backend='augsynth' is a migration bridge)	$\text{rel} \leq 1.7 \times 10^{-8}$	PASS
19	Generalized SCM (Xu 2017)	$\text{rel} \leq 7.7 \times 10^{-14}$ (native gsynth/fect two-way FE factor convention parity)	$\text{rel} \leq 2.2 \times 10^{-15}$	PASS
20	Goodman-Bacon decomposition	$\text{rel} \leq 5.6 \times 10^{-16}$ (R/Stata bacondecomp dyad parity)	$\text{rel} \leq 9.6 \times 10^{-9}$	PASS
21	Honest-DiD relative-mags	$\text{abs} \leq 4.44 \times 10^{-16}$ (native ARP conditional set; identical grid points)	$\text{abs} \leq 5.55 \times 10^{-17}$	PASS
22	sensemkr robustness	$\text{rel} \leq 5 \times 10^{-8}$ (sensemkr RV and benchmark bound-scale parity)	$\text{rel} \leq 5 \times 10^{-8}$	PASS

(continued on the next page)

(Table 17 continued)

ID	Method	Worst diff vs R	Worst diff vs Stata	Verdict
23	E-value (closed form)	$\text{rel} \leq 5.8 \times 10^{-14}$	$\text{rel} \leq 1.2 \times 10^{-16}$	PASS
24	Cox proportional hazards	$\text{rel} \leq 8.4 \times 10^{-16}$	$\text{rel} \leq 2.1 \times 10^{-10}$	PASS
25	Linear mixed model	$\text{rel} \leq 2.6 \times 10^{-12}$	$\text{rel} \leq 8.3 \times 10^{-11}$	PASS
26	GLMM logit (Laplace)	$\text{rel} \leq 0.00017$	$\text{rel} \leq 1.4 \times 10^{-8}$	PASS
27	GLMM logit (AGHQ, n=8)	$\text{rel} \leq 1.7 \times 10^{-7}$ (same 8-point AGQ likelihood (logLik rel 1×10^{-13}); intercept within reference optimiser scatter on a flat objective)	$\text{rel} \leq 9.8 \times 10^{-7}$	PASS
28	Stochastic frontier (cross-sec.)	$\text{rel} \leq 4.1 \times 10^{-8}$	$\text{rel} \leq 4 \times 10^{-8}$	PASS
29	Panel SFA (Pitt–Lee)	$\text{rel} \leq 2.8 \times 10^{-6}$ (slope parity; intercept/sigma rows are scale diagnostics)	$\text{rel} \leq 1.8 \times 10^{-6}$	PASS
30	Blinder–Oaxaca decomposition	$\text{rel} \leq 6.3 \times 10^{-16}$ (twofold vs threefold split convention)	$\text{rel} \leq 1.3 \times 10^{-16}$	PASS
31	DFL reweighting	$\text{rel} \leq 1.2 \times 10^{-9}$	$\text{rel} \leq 1.8 \times 10^{-13}$	PASS
32	RIF / UQR (median)	$\text{rel} \leq 2.2 \times 10^{-15}$	$\text{rel} \leq 1.4 \times 10^{-16}$	PASS
33	VAR (vars::VAR)	$\text{rel} \leq 3.1 \times 10^{-15}$	$\text{rel} \leq 6.6 \times 10^{-15}$	PASS
34	Local projections	$\text{rel} \leq 5 \times 10^{-15}$ (lpirfs Cholesky/unit-shock path (py-R) plus the direct-OLS Jorda regression (3-way); SEs are not joined on the direct rows because sp.local_projections applies Newey–West while lm/regress report classical OLS)	$\text{rel} \leq 4.4 \times 10^{-15}$	PASS
35	Panel FE/RE + Hausman [†]	$\text{rel} \leq 4.7 \times 10^{-14}$ (FE/RE and plm-style Hausman parity; Stata sigmamore diagnostic row)	$\text{rel} \leq 1.5 \times 10^{-15}$	PASS
36	Causal mediation (IKT)	$\text{rel} \leq 6.7 \times 10^{-15}$	$\text{rel} \leq 3.6 \times 10^{-15}$	PASS
37	PPML + HDFE	$\text{rel} \leq 4.9 \times 10^{-13}$ (post PPML FE-score fix; robust SE within 0.5% of fixest)	$\text{rel} \leq 2.2 \times 10^{-15}$	PASS
38	DR-DID (SZ 2020)	$\text{rel} \leq 2.6 \times 10^{-15}$ (panel DR-DID calibrated propensity parity with R/Stata)	$\text{rel} \leq 2.2 \times 10^{-16}$	PASS
39	ARIMA(2,0,0) [†]	$\text{rel} \leq 7.4 \times 10^{-7}$ (stats::arima ML / tightly converged Stata arima)	$\text{rel} \leq 9.3 \times 10^{-9}$	PASS
40	Quantile reg (median)	$\text{rel} \leq 3.3 \times 10^{-15}$ (post sqrt(n) sandwich-scaling fix)	$\text{rel} \leq 4.4 \times 10^{-15}$	PASS
41	Tobit (left-censored)	$\text{rel} \leq 2.8 \times 10^{-8}$ (post observed-information Hessian fix)	$\text{rel} \leq 2.8 \times 10^{-8}$	PASS
42	Negative binomial	$\text{rel} \leq 5.1 \times 10^{-10}$	$\text{rel} \leq 1 \times 10^{-11}$	PASS
43	Heckman 2-step	$\text{rel} \leq 1 \times 10^{-11}$	$\text{rel} \leq 1 \times 10^{-11}$	PASS
44	Multinomial logit	$\text{rel} \leq 2.6 \times 10^{-7}$ (post observed-information Hessian fix)	$\text{rel} \leq 1.5 \times 10^{-11}$	PASS
45	Ordered logit	$\text{rel} \leq 1.5 \times 10^{-7}$ (post observed-information Hessian fix)	$\text{rel} \leq 3.4 \times 10^{-7}$	PASS
46	Conditional logit	$\text{rel} \leq 1.3 \times 10^{-8}$	$\text{rel} \leq 1.3 \times 10^{-8}$	PASS

(continued on the next page)

(Table 17 continued)

ID	Method	Worst diff vs R	Worst diff vs Stata	Verdict
47	PPML + 3-way HDFE	rel $\leq 1.2 \times 10^{-15}$ (Post Gauss-Seidel multi-FE fix; sp matches at 1×10^{-15})	rel $\leq 1.2 \times 10^{-16}$	PASS
48	Binary probit	rel $\leq 3.1 \times 10^{-7}$	rel $\leq 1.6 \times 10^{-8}$	PASS
49	Ordered probit	rel $\leq 3.7 \times 10^{-7}$	rel $\leq 2.1 \times 10^{-10}$	PASS
50	Arellano-Bond GMM	rel $\leq 9 \times 10^{-16}$ (R/Stata dynamic-panel fixture; block-diagonal instruments and one-step GMM weights match)	rel $\leq 1.4 \times 10^{-15}$	PASS
51	Newey-West HAC OLS	rel $\leq 1.4 \times 10^{-15}$ (post HAC sandwich sqrt(n) scaling fix)	rel $\leq 8.5 \times 10^{-16}$	PASS
52	Classical SCM (unique solution)	rel $\leq 7.8 \times 10^{-8}$ (identified convex SCM; sp and Stata synth recover exact weights+gap)	rel $\leq 7.7 \times 10^{-8}$	PASS
53	Cluster-robust CR2 / CR3 SE	abs $\leq 1.84 \times 10^{-8}$ (CR2 and analytic CR3 match clubSandwich)	abs $\leq 5.93 \times 10^{-9}$	PASS
54	Two-way cluster-robust SE	abs $\leq 7.84 \times 10^{-16}$ (matches sandwich::vcovCL(HC1,cadjust); fixest min-G df convention differs $\sim 10^{-3}$)	abs $\leq 1.33 \times 10^{-15}$	PASS
55	HC2 / HC3 robust SE	abs $\leq 6.39 \times 10^{-9}$	abs $\leq 4.01 \times 10^{-13}$	PASS
56	Three-way cluster-robust SE	abs $\leq 2.09 \times 10^{-15}$	abs $\leq 7.77 \times 10^{-16}$	PASS
57	Binary logit ML	rel $\leq 2.7 \times 10^{-11}$	rel $\leq 2.7 \times 10^{-11}$	PASS
58	Poisson ML	rel $\leq 9.2 \times 10^{-15}$	rel $\leq 8.7 \times 10^{-12}$	PASS
59	LIML k-class IV	rel $\leq 1.7 \times 10^{-15}$ (ivregress runs with ‘small’ (RSS/(n-k)))	rel $\leq 9.7 \times 10^{-16}$	PASS
60	SUR one-step FGLS	rel $\leq 1.5 \times 10^{-14}$ (Sigma divisor n (sureg default / noDfCor))	rel $\leq 1.5 \times 10^{-15}$	PASS
61	Beta regression ML	rel $\leq 2.2 \times 10^{-12}$ (betareg SEs are expected-information (documented))	rel $\leq 2.1 \times 10^{-8}$	PASS
62	Truncated regression ML	rel $\leq 3.5 \times 10^{-10}$	rel $\leq 7.9 \times 10^{-8}$	PASS
63	Zero-inflated Poisson	rel $\leq 1.2 \times 10^{-7}$	rel $\leq 2.8 \times 10^{-8}$	PASS
64	Zero-inflated NB	rel $\leq 9.5 \times 10^{-7}$ (flat ZINB likelihood near optimum (1×10^{-5} budget))	rel $\leq 4.5 \times 10^{-11}$	PASS
65	Spatial ML (SAR/SEM/SDM)	rel $\leq 8.3 \times 10^{-8}$ (vs spatial-reg::lagsarlm/errorsarlm/Durbin, row-std rook W)	rel $\leq 5.1 \times 10^{-8}$	PASS
66	Spatial GMM (SAR-2SLS/SEM-GMM)	rel $\leq 4.6 \times 10^{-8}$ (vs spatial-reg::stsls(W2X=F)/GMerrorsar; SEM point-only)	rel $\leq 7.3 \times 10^{-16}$	PASS
67	Panel GLM (feglm / fepois) [†]	rel $\leq 9.7 \times 10^{-9}$ (vs fixest::feglm/fepois, absorbed id FE; IWLS SE 1×10^{-5})	rel $\leq 1.8 \times 10^{-9}$	PASS
68	Within transformation	rel $\leq 3.5 \times 10^{-15}$ (sp.demean(solver=’map’) vs textbook mean-within)	rel $\leq 8.8 \times 10^{-15}$	PASS
69	Panel balance filter	rel ≤ 0 (sp.balance_panel vs base R counts == n_periods)	rel ≤ 0	PASS

(continued on the next page)

(Table 17 continued)

ID	Method	Worst diff vs R	Worst diff vs Stata	Verdict
70	Policy tree (exact, depth 1–2)	$\text{rel} \leq 9.6 \times 10^{-16}$ (shared AIPW scores; exact welfare optimum vs policytree, all 1200 per-row policy decisions identical at both depths)	$\text{rel} \leq 1.4 \times 10^{-16}$	PASS
71	DML family (IRM / PLIV / IIVM)	$\text{rel} \leq 9.1 \times 10^{-11}$ (shared explicit fold_id across all three DoubleML model classes; PLIV at the floating-point floor, IRM/IIVM limited by the logistic-MLE optimiser)	$\text{rel} \leq 6.9 \times 10^{-7}$	PASS
72	TMLE (targeting step)	$\text{rel} \leq 1.9 \times 10^{-9}$ (shared initial Q / g1W; per-arm fluctuation matching tmle::tmle's submodel)	$\text{rel} \leq 1.9 \times 10^{-9}$	PASS
73	Gardner two-stage DiD	$\text{rel} \leq 4.8 \times 10^{-8}$ (estimate rel 4.8×10^{-8} and SE rel 2.7×10^{-10} against did2s (Stata did2s: 2.4×10^{-12} / 1.3×10^{-14}); vce='analytic' is the did2s corrected two-stage clustered variance, so stage-1 estimation error is propagated on all three sides)	$\text{rel} \leq 2.4 \times 10^{-12}$	PASS
74	Changes-in-Changes (ATT + QTE)	$\text{rel} \leq 2.4 \times 10^{-15}$ (machine precision vs qte::CiC (worst 4.4×10^{-15}))	$\text{rel} \leq 0.006$ (Stata convention gap)	PASS
75	Stacked DiD (never-treated controls)	$\text{rel} \leq 3.9 \times 10^{-13}$ (rel $< 1 \times 10^{-6}$ vs a hand-written fixest stack)	$\text{rel} \leq 7.1 \times 10^{-13}$	PASS
76	Pre-trends power (Roth 2022)	$\text{rel} \leq 4 \times 10^{-5}$ (rel $< 1 \times 10^{-4}$ vs pretrends::pretrends; the 1×10^{-3} budget covers mvtnorm::pmvnorm's randomised integrator)	$\text{rel} \leq 0.00014$	PASS
77	Triple differences ATT(g,t)	$\text{rel} \leq 5.7 \times 10^{-15}$ (machine precision vs tripleddiff::ddd (worst 1×10^{-12}), estimates and analytic SEs, across dr / ipw / reg)	<i>no Stata implementation: the Ortiz-Villavicencio/Sant'Anna DDD estimator ships as the R package tripleddiff. Re-checked 2026-08-06: neither 'tripleddiff' nor 'ddd' resolves locally and 'ssc describe tripleddiff' returns r(601).</i>	PASS
78	dCDH intertemporal event study	$\text{rel} \leq 3.3 \times 10^{-15}$ (machine precision vs DIDmultiplegtDYN (worst 5×10^{-15}), across absorbing and switch-off designs)	$\text{rel} \leq 2.1 \times 10^{-15}$	PASS
79	Functional-form test (Roth-Sant'Anna)	$\text{rel} \leq 1.3 \times 10^{-14}$ (machine precision vs didFF (worst 1.2×10^{-15}); the module's 1×10^{-3} budget covers only the simulated p-value)	<i>no Stata implementation: the Roth-Sant'Anna functional-form test ships as the R package didFF (GitHub only, not CRAN). Re-checked 2026-08-06: 'ssc describe didff' returns r(601), not found.</i>	PASS

(continued on the next page)

(Table 17 continued)

ID	Method	Worst diff vs R	Worst diff vs Stata	Verdict
80	Continuous-treatment DiD (CGS)	rel $\leq 2.4 \times 10^{-14}$ (machine precision vs <code>contdid::cont_did</code> (worst 1×10^{-12}))	<i>no Stata implementation: the CGS continuous-treatment estimator ships as the R package <code>contdid</code> (GitHub only, not CRAN). Re-checked 2026-08-06: ‘<code>ssc describe contdid</code>’ returns <code>r(601)</code>, not found.</i>	PASS
81	dCDH 2020 DID_M (on/off switching)	rel $\leq 3.9 \times 10^{-15}$ (machine precision vs archived DIDmultiplegt 0.1.4)	rel $\leq 3.2 \times 10^{-15}$	PASS
82	Design-based staggered rollout (Roth-Sant’Anna)	rel $\leq 9.1 \times 10^{-16}$ (machine precision on all 33 rows (worst 3.7×10^{-15}) against both staggered 1.2.2 in R and the SSC staggered port in Stata, including both the Neyman and adjusted standard errors)	rel $\leq 5.7 \times 10^{-16}$	PASS
83	LP-DiD event study (Dube-Girardi-Jorda-Taylor)	rel $\leq 5 \times 10^{-15}$ (estimate and SE both rel $< 1 \times 10^{-14}$ vs a direct transcription)	rel $\leq 2.5 \times 10^{-15}$	PASS
84	BJS pre-treatment lead vector (Borusyak-Jaravel-Spiess)	rel $\leq 5.1 \times 10^{-8}$ (point estimates rel $< 1 \times 10^{-7}$ against both references; the three leads match Stata <code>pretrends(3)</code> at rel $< 1 \times 10^{-14}$ on estimate and SE, and are a py $\leftrightarrow$ Stata pin because R exposes no equivalent option; horizon SEs are an open gap)	rel $\leq 5.1 \times 10^{-10}$	PASS
85	Dynamic TWFE event study (benchmark specification)	rel $\leq 3.2 \times 10^{-13}$ (estimates three-way to 5.7×10^{-14} ; SE matches <code>fixest</code> to 9.3×10^{-14} under <code>ssc(fixef.K="none")</code> , and <code>reghdfe</code> ’s SE is reproduced from it by the exact $(N - K_{py})/(N - K_{St})$ d.f. scalar)	rel $\leq 1.5 \times 10^{-14}$	PASS
86	fect counterfactual estimators (fe / ife / mc)	rel $\leq 1.8 \times 10^{-13}$ (native port of fect’s EM map; three outcome models on one staggered panel)	rel $\leq 9.8 \times 10^{-10}$	PASS
87	interflex marginal effects (linear / binning / kernel)	rel $\leq 4 \times 10^{-15}$ (native port incl. R’s <code>density()</code> grid for the adaptive kernel bandwidth)	rel $\leq 1.5 \times 10^{-14}$	PASS
88	RD bandwidth selection (all ten CCT selectors)	rel $\leq 9.4 \times 10^{-13}$ (all 68 bandwidths agree with <code>rdrobust</code> to 1.8×10^{-12} and with Stata to 3.7×10^{-9} ; <code>certwo</code> is pinned against R only because Stata <code>rdbwselect</code> 10.0.0 exits <code>r(3200)</code> on that selector, including on the package authors’ own <code>rdrobust_senate.dta</code>)	rel $\leq 3.5 \times 10^{-9}$	PASS

(continued on the next page)

(Table 17 continued)

ID	Method	Worst diff vs R	Worst diff vs <i>Stata</i>	Verdict
89	Multi-score / geographic RD at boundary points	rel $\leq 1.2 \times 10^{-12}$ (three boundary points along one boundary; bias-corrected and conventional estimates, standard errors, selected bandwidths and per-side effective sample sizes agree with rdmulti to 7.6×10^{-12} and with <i>Stata</i> to 3.3×10^{-9} , and the six effective sample sizes are exactly equal on all three sides)	rel $\leq 5.4 \times 10^{-10}$	PASS

Table 17: Every Track A module, at **StatsPAI** 1.30.1. The ID column is the module’s two-digit prefix in `tests/r_parity/`. The two difference columns report the worst residual gap across that module’s headline rows, point estimates only; per-row standard errors, confidence bounds, and documented convention gaps are in the generated `tests/r_parity/results/parity_table_3way.md`. Italic text in the *Stata* column is the measured reason no portable *Stata* artifact exists for that module, not an untested assumption of unavailability. All 89 modules are joined to a canonical R reference and 85 additionally to a canonical or audited *Stata* bridge, every side reading the same CSV bytes.

Of the 89 modules, all but one receive a pass-type verdict. A pass-type verdict covers ordinary strict passes, common-specification passes such as RD at a forced common bandwidth, seeded stochastic passes such as DML, and coefficient-level passes whose secondary diagnostic follows a documented small-sample convention. The single remaining *GAP* is the classical-SCM donor-weight non-uniqueness disclosure of Section 5.4, kept visible rather than absorbed into a wider tolerance.

Two things this table does not do: it omits per-row detail, which is regenerated into `tests/r_parity/results/parity_table_3way.md`, and it does not by itself show that either side is re-derivable (Section 8.2).

B. Output-changing corrections

Table 18 lists the output-changing fixes that affect interpretation of the present validation claim; each is pinned by a regression test, and users on affected versions should re-estimate. Section 5.10 discusses the 1.23.0 and 1.25.0 entries; none of the 174 defects of 1.29.0 had been caught by a unit test.

Affiliation:

Biaoyue Wang (corresponding author)

Rural Education Action Program, Stanford Center on China’s Economy and Institutions,
Stanford University, United States

and StatsPAI Inc., United States

E-mail: brycew6m@gmail.com, brycew6m@stanford.edu

URL: <https://github.com/brycewang-stanford/StatsPAI>

Version	Affected area	Output-changing fix
1.6–1.15	IV/LIML/Heckman, event study, CS-DiD, TMLE, DML, metalearners, mediation	Small-sample, two-step and rank SE corrections; influence-function, orthogonal-score and AIPW fixes.
1.16.0	Quantile regression, Newey–West, Arellano–Bond; forest ATE/ATT	SE scaling and GMM weight construction corrected; forest headline moved to the grf AIPW estimand.
1.21.0	CBPS, entropy balancing, Mahalanobis, genetic matching	GMM basis, exact moment balance, and covariance conventions corrected (up to $8.6\times$ off CBPS); now pinned against CBPS , ebal , MatchIt , optmatch , Matching , sbw .
1.23.0	CS-DiD unit weights; IPW SEs	Weighted estimand restored; IPW SEs were 9–11% (up to 89% weighted) too large; pinned to did::att_gt (see text).
1.24.0	Sun–Abraham, GLMM, PPML-HDFE, dynamic TWFE SEs	Degrees-of-freedom/nesting conventions fixed to the reference rules; SEs moved 0.1–2% (see text).
1.25.0	Causal-forest aggregation under a continuous treatment	The binary AIPW score was applied to a continuous treatment (-1266.6 , $p = 0.0000$ on Card); guarded in 1.25.0, replaced by the Riesz-representer average in 1.29.0. Binary results unchanged.
1.26.0	Evidence grades	A false alias certification (sp.wooldridge_did as sp.etwfe) withdrawn; grades, not estimates, changed.
1.27.0	ETWFE, sp.wooldridge_did , sp.etregress , RE panel logit/probit, panel FGLS, spatial LM tests	Cohort ATTs read from a saturated regression; clustered SEs account for absorbed effects; the etregress default is a true MLE with first-stage-aware two-step SEs; lm_tests no longer reverses its model-selection conclusion.
1.28.0	SCM placebo p -values; Mendelian randomization; evidence grades	Permutation p -values now rank the treated unit with its placebos, $(1 + \#\{\text{placebo} \geq \text{treated}\})/(J + 1)$; MR estimators ported to their reference definitions; 31 functions graded T2 on closed-form tests that never ran R or Stata were regraded T1 (see text).
1.29.0	Cross-language parity campaign (174 defects) and Track A closure	Fourteen families bisected against their reference on identical bytes; 174 output-changing defects fixed, none caught by a unit test; forest defaults follow grf ; Gardner two-stage SE as did2s ; PSM SE as teffects psmatch .
1.30.0	HonestDiD on a raw CS fit; forest ATT/ATC under a continuous treatment	The event-study covariance passed to sp.honest_did dropped the cohort-share term off the diagonal (cross-cell entries up to 8% off did), overstating robustness; now matches did to 2.8×10^{-13} , and the bounds of Listing 6 moved in the fourth decimal. ATT and ATC now raise for a continuous treatment, as in grf .
1.31.0	Entropy-balancing SE; audit applicability; Honest-DiD relative magnitudes; evidence grades	sp.ebalance SE was twice the Monte Carlo SD; now the M-estimation sandwich (WeightIt to 3×10^{-9}). sp.audit : over-identification test not applicable on just-identified fits. sp.causal_question forest design reports the forest’s own AIPW average. Native relative-magnitudes Honest-DiD set; two parity rows that ran R itself rebuilt on native code; forest T3 re-established by seed replication.

Table 18: Output-changing correctness fixes through release 1.30.1; CHANGELOG and MIGRATION.md hold the detail.

Scott Rozelle

Rural Education Action Program, Stanford Center on China’s Economy and Institutions,
Stanford University, United States

and StatsPAI Inc., United States

E-mail: rozelle@stanford.edu

Journal of Statistical Software

published by the Foundation for Open Access Statistics

MMMMMM YYYY, Volume VV, Issue II

[doi:10.18637/jss.v000.i00](https://doi.org/10.18637/jss.v000.i00)

<https://www.jstatsoft.org/>

<https://www.foastat.org/>

Submitted: yyyy-mm-dd

Accepted: yyyy-mm-dd